Lecture 5: DALL-E and Score-Based Generative Models

텍스트-이미지 자기회귀 생성과 스코어 기반 생성 모델의 기초

목차 (Table of Contents)

- 0. 개요 (Overview)
  - 0.1 선수 지식 (Prerequisites)
  - 0.2 학습 목표 (Learning Objectives)
- 1. DALL-E
  - 1.1 핵심 아이디어
  - 1.2 Stage 1: dVAE (Discrete VAE) 훈련
    - VQ-VAE vs dVAE의 코드 선택 방식
    - 이미지 압축
    - Stage 1 완료 후 필수 작업
    - PyTorch 구현: dVAE Quantization
  - 1.3 Stage 2: 트랜스포머 디코더 훈련
    - 텍스트와 이미지 토큰의 연결
    - 훈련 과정 (Teacher Forcing)
    - 로스 계산 범위
    - 추론 (생성) 과정
    - 같은 프롬프트에서 다양한 이미지가 나오는 이유
    - PyTorch 구현: Transformer Decoder for Autoregressive Generation
  - 1.4 DALL-E의 생성 결과
  - 1.5 CogView / Crayon: 인코더-디코더 구조
- 2. Score-Based Generative Models
  - 2.1 기존 생성 모델의 한계
  - 2.2 핵심 발상의 전환: 확률 대신 기울기를 학습
    - 스코어 함수 (Score Function) 정의
    - 스코어의 의미
    - 중력 비유
    - 벡터 필드 (Vector Field)
  - 2.3 스코어 계산 예시: 가우시안 분포
  - 2.4 스코어의 장점: 정규화 불필요
  - 2.5 스코어 매칭 (Score Matching)
    - 목표
    - 훈련 목표 (이상적)

        - 부분 적분을 통한 변환
        - 계산량 문제와 Sliced Score Matching
        - PyTorch 구현: Score Matching Loss
    - 2.6 Langevin Dynamics: 스코어로부터 샘플링
        - PyTorch 구현: Langevin Dynamics Sampling Loop
    - 2.7 데이터 희소 영역 문제와 Noise Perturbation
    - 2.8 생성 과정 요약
- 3. 핵심 수식 총정리
- 4. 이후 학습 로드맵
    - 4.1 반드시 공부해야 할 핵심 모델들
    - 4.2 기반 지식으로 필요한 것들
    - 4.3 응용 분야
    - 4.4 추천 학습 순서
- 5. 참고문헌 (References)

---

0. 개요

이번 강의에서는 VQ-GAN의 구조를 텍스트-이미지 생성으로 확장한 **DALL-E**와, 현재 생성 모델의 주류를 이루는 **Score-Based Generative Model**의 기본 개념을 다룬다.

> 💡 **왜 이 두 주제를 함께 다루는가?** DALL-E와 Score-Based Model은 생성 모델의 **두 가지 거대한 흐름**을 대표한다. DALL-E는 "이미지를 이산 토큰으로 바꿔서 언어 모델처럼 생성하자"는 **자기회귀(autoregressive) 패러다임**이고, Score-Based Model은 "확률 분포의 기울기를 따라가며 노이즈에서 이미지를 만들자"는 **연속적 디퓨전 패러다임**이다. 2021년에는 DALL-E 방식이 주목받았지만, 이후 Score-Based/Diffusion 방식이 품질과 효율 면에서 압도적 우위를 점하면서 Stable Diffusion, DALL-E 2/3, Midjourney 등 거의 모든 최신 모델의 기반이 되었다. 두 접근법의 철학적 차이와 각각의 장단점을 이해하는 것이 현대 생성 모델 전체를 조감하는 데 핵심적이다.

모델	연도	핵심
DALL-E	2021.01	텍스트 → 이미지 자기회귀 생성 (OpenAI)
CogView / Crayon	2021.07	DALL-E 변형, 인코더-디코더 구조
Score-Based Model	2019~	확률 분포 대신 **스코어(기울기)**를 학습

0.1 선수 지식 (Prerequisites)

이 강의를 이해하기 위해 다음 개념들에 대한 사전 지식이 필요하다:

분류	필요 지식	관련 강의
딥러닝 기초	CNN, Transformer (Self-Attention, Cross-Attention), Autoregressive Models	강의 1-2
생성 모델	VAE, VQ-VAE, VQ-GAN의 구조 및 학습 원리	강의 3-4
수학	확률 분포, 로그 우도, 그래디언트(편미분), 야코비안 (Jacobian)	-
프레임워크	PyTorch 기본 사용법 (텐서 연산, `nn.Module`, Autograd)	-

> ⚠️ **주의**: VQ-VAE와 VQ-GAN의 코드북(codebook) 개념을 명확히 이해하지 못한 경우, 강의 3-4를 복습한 후 진행하는 것을 권장한다. DALL-E의 dVAE는 VQ-VAE의 변형이므로 기초 개념이 필수적이다.

0.2 학습 목표 (Learning Objectives)

이 강의를 마치면 다음을 할 수 있어야 한다:

1. **DALL-E의 2-Stage 구조**를 설명하고, dVAE와 VQ-VAE의 차이를 비교할 수 있다
2. **텍스트-이미지 토큰 연결** 방식과 자기회귀 생성 과정을 이해한다
3. **디코더-only vs 인코더-디코더** 구조의 장단점을 설명할 수 있다
4. **스코어 함수(Score Function)**의 정의와 의미를 설명할 수 있다
5. **스코어 매칭(Score Matching)** 목적 함수를 유도하고, 계산 가능한 형태로 변환하는 과정을 이해한다
6. **Langevin Dynamics**를 이용한 샘플링 과정을 구현할 수 있다
7. **다중 노이즈 스케일(Multi-Scale Noise Perturbation)** 기법의 필요성과 원리를 이해한다
8. Score-Based Model이 이후 **DDPM, Stable Diffusion** 등으로 어떻게 발전하는지 로드맵을 그릴 수 있다

---

1. DALL-E

> 📄 **논문**: Ramesh et al., "*Zero-Shot Text-to-Image Generation*", OpenAI, 2021.01
>
> 이름 유래: 초현실주의 화가 Salvador **Dalí** + 애니메이션 로봇 W**ALL-E**

1.1 핵심 아이디어

> 💡 **왜 이미지를 "토큰"으로 바꾸는가?** 2020-2021년 시점에서 Transformer는 자연어 처리(NLP)에서 압도적인 성능을 보여주고 있었다. GPT-2, GPT-3의 성공은 "충분히 큰 자기회귀 Transformer에 충분히 많은 데이터를 넣으면 놀라운 생성 능

> 력이 나온다"는 것을 증명했다. 그렇다면 자연스러운 질문은: **이미지도 토큰의 시퀀스로 표현할 수 있다면, 같은 Transformer 프레임워크로 이미지를 생성할 수 있지 않을까?** 이것이 DALL-E의 핵심 발상이다. 이미지를 이산적 토큰(discrete token)으로 변환하면, 텍스트 토큰과 이미지 토큰을 하나의 시퀀스로 연결하여 "텍스트 다음에 이미지가 올 확률"을 언어 모델처럼 학습할 수 있다.

DALL-E는 VQ-GAN과 **거의 동일한 구조**이다:

- **Stage 1**: 이미지를 이산적 토큰으로 압축하는 오토인코더 훈련 (dVAE 사용)
- **Stage 2**: 트랜스포머 디코더로 이미지 토큰을 자기회귀적 생성
- **차이점**: 조건으로 **자연어 텍스트**를 사용 → Text-to-Image 생성

> 💡 **왜 2-Stage 구조인가?** 이미지를 픽셀 단위로 자기회귀 생성하면 $256 \times 256 \times 3 = 196{,}608$개의 값을 순차적으로 예측해야 한다. 이는 Transformer의 self-attention이 시퀀스 길이의 제곱에 비례하는 계산량($O(n^2)$)을 가지기 때문에 사실상 불가능하다. Stage 1에서 이미지를 1,024개의 토큰으로 **압축**하면, Transformer가 처리해야 하는 시퀀스 길이가 약 200배 줄어든다. 이것이 "먼저 압축하고, 그 다음에 생성한다"는 2-Stage 전략의 핵심 이유이다. VQ-GAN(강의 4)에서도 동일한 전략을 사용했지만, DALL-E는 여기에 **텍스트 조건**을 추가한 것이다.

스펙:

- 파라미터: **120억 개**
- 훈련 데이터: **2억 5천만 개** 텍스트-이미지 쌍
- Zero-shot: 훈련 때 보지 못한 텍스트에 대해서도 이미지 생성 가능

> ⚠️ **중요**: DALL-E는 2021년 기준으로 매우 대규모 모델이었으며, 이 규모의 학습에는 수백 개의 GPU와 수 주 이상의 훈련 시간이 필요했다. 학습 인프라의 확보가 이 연구의 핵심 도전 과제 중 하나였다.

1.2 Stage 1: dVAE (Discrete VAE) 훈련

> 💡 **왜 VAE가 아닌 "Discrete" VAE인가?** 일반 VAE(Variational Autoencoder)는 잠재 변수 z가 **연속적(continuous)**이다 -- 예를 들어 $z \in \mathbb{R}^{256}$처럼 실수 벡터이다. 그런데 Stage 2에서 Transformer로 자기회귀 생성을 하려면, 각 위치에서 "다음 토큰의 확률 분포"를 소프트맥스로 출력해야 한다. 소프트맥스는 **유한한 개수의 선택지** 위에서만 정의되므로, 잠재 표현이 반드시 **이산적(discrete)**이어야 한다. 연속 벡터 $z \in \mathbb{R}^{256}$에 대해서는 소프트맥스를 쓸 수 없다. 즉, Transformer 자기회귀 생성이라는 Stage 2의 요구사항이 Stage 1에서 이산 코드북을 강제하는 것이다. 이것이 VQ-VAE/dVAE가 필요한 근본적 이유이다.

DALL-E는 VQ-VAE 대신 **dVAE**를 사용한다.

VQ-VAE vs dVAE의 코드 선택 방식

VQ-VAE	dVAE (DALL-E)

코드 선택	L2 거리가 가장 가까운 코드	**소프트맥스 확률**이 가장 높은 코드
코드북 크기	1,024개	**8,192개**
수식	$k = \arg\min_j \lvert z_e - e_j \rvert_2$	$k = \arg\max_j \mathrm{softmax}(f(z_e))_j$

dVAE에서는 인코더가 8,192개 코드에 대한 **확률 분포를 출력**하고, 가장 확률이 높은 코드를 선택한다. 방법만 다를 뿐 본질은 동일하다.

> 💡 **왜 VQ-VAE 대신 dVAE를 선택했는가?** VQ-VAE는 L2 거리 기반의 `argmin`으로 코드를 선택하고, straight-through estimator로 그래디언트를 전달한다. 이 방식은 잘 작동하지만 두 가지 한계가 있다: (1) **코드북 활용률(codebook utilization)** 문제 -- 일부 코드만 반복 사용되고 나머지는 "죽는(dead code)" 현상이 빈번하다. L2 거리 기반 선택은 초기화에 민감하고 특정 코드로 편향되기 쉽다. (2) **확률적 해석의 부재** -- VQ-VAE의 코드 선택은 확정적(deterministic)이므로, VAE의 확률적 프레임워크(ELBO 최적화)와 이론적으로 잘 맞지 않는다. dVAE는 각 코드에 대한 **확률 분포**를 명시적으로 모델링하므로, 정보 이론적으로 더 깔끔하고 코드북 전체를 골고루 활용하도록 유도할 수 있다. 8,192개라는 큰 코드북을 효과적으로 사용하려면 이러한 확률적 접근이 유리하다.

> ⚠️ **dVAE의 Gumbel-Softmax Trick**: dVAE에서 `argmax`는 미분 불가능하므로, 훈련 시에는 **Gumbel-Softmax relaxation**을 사용하여 연속적 근사를 수행한다. 온도(temperature) 파라미터 τ를 점차 낮추면서 이산 선택에 가까워지도록 annealing한다.

Gumbel-Softmax가 필요한 이유와 작동 원리를 깊이 이해하자:

핵심 문제는 이것이다: 우리는 확률 분포 $\pi = \mathrm{softmax}(f(z_e))$로부터 이산적으로 하나의 코드를 **샘플링**해야 한다. 그런데 "샘플링"이라는 연산은 미분이 불가능하다. 주사위를 던져서 3이 나왔다고 할 때, "3이 나온 것"을 주사위의 확률에 대해 미분할 수 없다.

Forward pass (순전파): Gumbel-Softmax는 다음과 같이 작동한다:

$$y_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum_{j=1}^{K} \exp((\log \pi_j + g_j)/\tau)}$$

여기서 $g_i = -\log(-\log(u_i))$는 Gumbel 노이즈이고, $u_i \sim \mathrm{Uniform}(0, 1)$이다. 온도 $\tau \to 0$이면 이 결과는 one-hot 벡터에 수렴하고(즉, argmax와 동일), $\tau \to \infty$이면 균등 분포에 수렴한다.

Straight-Through Estimator: 실제 구현에서는 `hard=True` 옵션을 사용한다:

- Forward에서는 실제로 argmax를 수행하여 **진짜 one-hot 벡터**를 만든다
- Backward에서는 argmax를 무시하고, 위의 연속적 softmax 근사의 그래디언트를 사용한다
- 코드로 표현하면: `y_hard - y_soft.detach() + y_soft` -- 이 트릭으로 forward 값은 `y_hard`이지만, 그래디언트는 `y_soft`를 통해 흐른다

이 방식으로 "이산적 선택"과 "미분 가능성"이라는 두 마리 토끼를 동시에 잡는다.

이미지 압축

```
256x256 이미지 → [CNN Encoder] → 32x32 피처 맵 → [Quantize: 8,192개 중
```

- 입력: 256×256 이미지
- 출력: $32 \times 32 = 1{,}024$개의 이미지 토큰
- 각 토큰은 8,192개의 코드 중 하나 → **8,192가 이미지 vocabulary 크기**

압축률을 구체적으로 이해하자: 원본 이미지는 $256 \times 256 \times 3 = 196{,}608$개의 값(RGB 채널)으로 구성된다. 이를 1,024개의 정수 인덱스(0~8,191)로 표현하면, 정보량 기준으로 $1{,}024 \times \log_2(8{,}192) = 1{,}024 \times 13 = 13{,}312$ 비트이다. 원본이 8비트 기준 $196{,}608 \times 8 = 1{,}572{,}864$ 비트이므로, 약 **118배** 압축이다. 당연히 손실(lossy) 압축이며, dVAE 디코더가 이 1,024개 토큰으로부터 원본에 가까운 이미지를 복원할 수 있도록 학습된다. 압축률이 높을수록 Stage 2 Transformer의 부담은 줄지만, 복원 품질이 떨어지므로 **압축률과 복원 품질 사이의 트레이드오프**가 존재한다.

Stage 1 완료 후 필수 작업

훈련이 끝나면, **2억 5천만 장의 이미지 각각에 대해** 코드북 인덱스 시퀀스를 추출하여 저장:

$$\text{모든 이미지 } x_i \xrightarrow{\text{dVAE Encoder + Quantize}} \text{이미지 토큰 시퀀스 } s_i \quad (|\, s_i \,| = 1{,}024)$$

PyTorch 구현: dVAE Quantization

아래는 dVAE의 핵심인 소프트맥스 기반 코드 선택과 Gumbel-Softmax를 활용한 양자화 과정의 PyTorch 구현이다:

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class dVAEQuantizer(nn.Module):
    """
    dVAE 양자화 모듈: 소프트맥스 확률 기반 코드 선택
    VQ-VAE의 L2 거리 기반 선택과 달리, 확률 분포를 출력하고 Gumbel-Soft
    """
    def __init__(self, num_codes=8192, code_dim=256, temperature=1.(
        super().__init__()
        self.num_codes = num_codes       # 코드북 크기 (DALL-E: 8,19.
        self.code_dim = code_dim         # 각 코드 벡터의 차원
        self.temperature = temperature   # Gumbel-Softmax 온도 파라(

        # 코드북: 8,192개의 학습 가능한 코드 벡터
        self.codebook = nn.Embedding(num_codes, code_dim)

        # 인코더 출력 → 코드 확률을 계산하기 위한 선형 변환
        # (피처 벡터를 num_codes 차원의 로짓으로 변환)
        self.logit_proj = nn.Linear(code_dim, num_codes)

    def forward(self, z_e):
        """
        Args:
            z_e: 인코더 출력 피처 맵, shape = (B, code_dim, H, W)
                 예: (batch_size, 256, 32, 32)
        Returns:
            z_q: 양자화된 피처 맵, shape = (B, code_dim, H, W)
            indices: 선택된 코드북 인덱스, shape = (B, H*W)
            logits: 코드 선택 로짓, shape = (B, H*W, num_codes)
        """
        B, C, H, W = z_e.shape

        # (B, C, H, W) → (B, H*W, C): 공간 차원을 시퀀스로 변환
        z_e_flat = z_e.permute(0, 2, 3, 1).reshape(B, H * W, C)

        # 각 위치에서 8,192개 코드에 대한 로짓 계산
        logits = self.logit_proj(z_e_flat)  # (B, H*W, num_codes)

        if self.training:
            # 훈련 시: Gumbel-Softmax로 미분 가능한 "소프트" 선택
            # (straight-through estimator 사용: forward는 hard, back
            soft_one_hot = F.gumbel_softmax(
                logits, tau=self.temperature, hard=True, dim=-1
            )  # (B, H*W, num_codes) - hard one-hot이지만 gradient는
        else:
            # 추론 시: 가장 확률 높은 코드를 직접 선택 (argmax)
            indices_hard = logits.argmax(dim=-1)  # (B, H*W)
            soft_one_hot = F.one_hot(
                indices_hard, self.num_codes
            ).float()  # (B, H*W, num_codes)
```

```python
        # 선택된 코드 인덱스 추출
        indices = soft_one_hot.argmax(dim=-1)  # (B, H*W)

        # 코드북에서 벡터 복원: one-hot @ codebook_weight
        z_q = torch.matmul(
            soft_one_hot, self.codebook.weight
        )  # (B, H*W, code_dim)

        # 다시 공간 차원으로 복원: (B, H*W, C) → (B, C, H, W)
        z_q = z_q.reshape(B, H, W, C).permute(0, 3, 1, 2)

        return z_q, indices, logits


class dVAE(nn.Module):
    """
    dVAE 전체 모델: CNN Encoder + dVAE Quantizer + CNN Decoder
    256x256 이미지를 32x32 = 1,024개 토큰으로 압축
    """
    def __init__(self, num_codes=8192, code_dim=256):
        super().__init__()

        # CNN 인코더: 256x256 → 32x32 피처 맵 (8배 다운샘플)
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 64, 4, stride=2, padding=1),    # 128x128
            nn.ReLU(),
            nn.Conv2d(64, 128, 4, stride=2, padding=1),  # 64x64
            nn.ReLU(),
            nn.Conv2d(128, 256, 4, stride=2, padding=1), # 32x32
            nn.ReLU(),
            nn.Conv2d(256, code_dim, 1),                  # 32x32, (
        )

        # dVAE 양자화 모듈
        self.quantizer = dVAEQuantizer(num_codes, code_dim)

        # CNN 디코더: 32x32 → 256x256 복원 (8배 업샘플)
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(code_dim, 256, 4, stride=2, padding=
            nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, stride=2, padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 3, 1),
            nn.Sigmoid(),  # [0, 1] 범위의 이미지 출력
        )

    def encode(self, x):
        """이미지 → 코드북 인덱스 시퀀스 추출 (Stage 2 데이터 준비용)""
        z_e = self.encoder(x)
```

```
        z_e = self.encoder(x)
        _, indices, _ = self.quantizer(z_e)
        return indices  # (B, 1024) — 이미지당 1,024개 토큰

    def forward(self, x):
        """전체 forward pass: 이미지 → 인코딩 → 양자화 → 복원"""
        z_e = self.encoder(x)
        z_q, indices, logits = self.quantizer(z_e)
        x_recon = self.decoder(z_q)
        return x_recon, indices, logits


# === 사용 예시 ===
if __name__ == "__main__":
    model = dVAE(num_codes=8192, code_dim=256)

    # 임의의 256x256 이미지 배치
    images = torch.randn(4, 3, 256, 256)

    # Forward pass
    recon, indices, logits = model(images)
    print(f"복원 이미지 shape: {recon.shape}")      # (4, 3, 256, 25
    print(f"토큰 인덱스 shape: {indices.shape}")     # (4, 1024)
    print(f"로짓 shape: {logits.shape}")             # (4, 1024, 819
    print(f"코드북 인덱스 범위: [{indices.min()}, {indices.max()}]")
```

> ⚠️ **Gumbel-Softmax 온도 스케줄링**: 실제 학습에서는 temperature를 초기값 1.0에서 점차 줄여 0.0625 수준까지 annealing한다. 온도가 높으면 코드 선택이 부드럽지만 부정확하고, 온도가 낮으면 이산적 선택에 가까워지지만 그래디언트가 불안정해진다.

1.3 Stage 2: 트랜스포머 디코더 훈련

> 💡 **왜 자기회귀(Autoregressive) 방식인가?** Stage 1에서 이미지를 1,024개의 이산 토큰으로 변환했다. 이제 이 토큰들의 **결합 확률 분포** $p(s_1, s_2, \ldots, s_{1024}|\text{text})$를 모델링해야 한다. 이 분포를 직접 모델링하는 것은 불가능하다 -- 각 토큰이 8,192개의 값을 가질 수 있으므로 가능한 조합은 $8{,}192^{1{,}024}$개로, 우주의 원자 수보다 압도적으로 많다. 자기회귀 분해(autoregressive factorization)는 이 문제를 우아하게 해결한다: $p(s_1, \ldots, s_{1024}) = \prod_{j=1}^{1024} p(s_j|s_{<j}, \text{text})$. 각 스텝에서는 8,192개짜리 소프트맥스 하나만 계산하면 된다. GPT가 언어에서 성공한 것과 정확히 같은 원리이다.

텍스트와 이미지 토큰의 연결

텍스트 토큰과 이미지 토큰을 **단일 시퀀스로 연결(concatenate)**:

$$[\text{텍스트 토큰}_1, \ldots, \text{텍스트 토큰}_m \mid \text{이미지 토큰}_1, \ldots, \text{이미지 토큰}_{1024}]$$

- 텍스트 vocabulary와 이미지 vocabulary는 **구분**되어 있음
- 텍스트: ~30,000개 vocabulary (BPE 등)
- 이미지: 8,192개 vocabulary (코드북)
- 번호 체계를 분리하거나, 로스 계산 시 구분 가능하도록 설계

BPE(Byte Pair Encoding) 토크나이저란? 텍스트를 토큰으로 변환하는 알고리즘이다. 단어 단위로 쪼개면 "unseen" 같은 새로운 단어를 처리할 수 없고, 글자 단위로 쪼개면 시퀀스가 너무 길어진다. BPE는 이 사이의 절충안으로, 자주 등장하는 문자 조합을 하나의 토큰으로 병합한다. 예: "banana" → ["ban", "ana"] 또는 ["b", "anana"]. 이렇게 하면 약 30,000~50,000개의 서브워드(subword) 토큰으로 거의 모든 영어 텍스트를 표현할 수 있다. DALL-E에서 텍스트 토큰의 최대 길이는 256개로 제한된다.

> ⚠️ **Vocabulary 분리**: 실제 구현에서는 텍스트 토큰 인덱스(0~29,999)와 이미지 토큰 인덱스(30,000~38,191)처럼 offset을 두어 하나의 임베딩 테이블에서 관리하거나, 별도의 임베딩 레이어를 사용한다. 혼합하면 학습이 불안정해질 수 있으므로 주의가 필요하다.

훈련 과정 (Teacher Forcing)

"Yellow bunch of bananas" + 바나나 이미지 토큰 시퀀스가 쌍으로 주어질 때:

```
입력 (디코더):  [yellow, bunch, of, bananas, y₁(정답), y₂(정답), y₃(정
목표 (예측):                                 [y₁,       y₂,
```

각 위치에서:

1. 소프트맥스 분포 출력 (8,192차원)
2. 정답 (원핫 벡터)과 **Cross-Entropy Loss** 계산
3. **Causal Mask** 적용: y_3 예측 시 y_4 이후는 보지 못함

Causal Mask 구현: 미래 토큰의 어텐션 로짓을 $-\infty$로 설정 → 소프트맥스 후 어텐션 가중치 = 0

$$\text{Attention}(i,j) = \begin{cases} \text{softmax}(QK^\top/\sqrt{d}) & \text{if } j \le i \\ 0 & \text{if } j > i \end{cases}$$

> 💡 **왜 Causal Mask가 필수적인가?** 자기회귀 모델의 핵심 가정은 $p(s_j|s_{<j})$, 즉 **과거 토큰만 보고 다음 토큰을 예측**하는 것이다. 만약 s_3를 예측할 때 s_4를 볼 수 있다면, 모델은 "미래를 컨닝"하여 훈련 로스는 낮지만 실제 생성 시에는 미래 토큰이 없으므로 성능이 크게 떨어진다 (train-test mismatch). Causal Mask는 attention score 계산 시 미래 위치의 로짓에 $-\infty$를 더하여, $\text{softmax}(-\infty) = 0$이 되도록 만든다. 이렇게 하면 각 위치는 자신과 이전 위치만 참조하게 된다.

> **참고**: 훈련 시에는 Teacher Forcing으로 모든 위치의 출력을 **동시에** 계산. 생성 시에만 순차적으로 생성. Teacher Forcing의 핵심 이점은 **병렬성**이다: Causal Mask 덕분에 위치 1, 2, ..., N의 예측을 하나의 forward pass로 동시에 계산할 수 있다. 생

> 성 시에는 s_{j+1}을 예측하려면 s_j가 필요하므로 병렬화가 불가능하고, 1,024번의 순차적 forward pass가 필요하다. 이것이 자기회귀 모델의 근본적 속도 한계이며, 이후 Diffusion 모델이 각광받게 된 이유 중 하나이다.

로스 계산 범위

이미지 토큰 예측 로스뿐만 아니라, **텍스트 토큰 영역의 다음 토큰 예측 로스도 함께 계산**한다:

$$\mathcal{L} = -\sum_i \log p_\theta(t_i \mid t_{<i}) - \sum_j \log p_\theta(s_j \mid t_{1:M}, s_{<j})$$

핵심은 이미지 토큰 예측이지만, 텍스트 영역 로스도 포함하여 전체적으로 훈련.

> 💡 **왜 텍스트 영역의 로스도 함께 계산하는가?** 순수하게 이미지 생성만이 목적이라면 이미지 토큰 로스만 쓰면 될 것 같지만, 텍스트 로스를 함께 쓰는 데에는 중요한 이유가 있다. Transformer의 텍스트 영역 hidden state는 이미지 토큰 예측의 **조건(condition)**으로 사용된다. 텍스트 다음 토큰 예측 로스를 함께 학습하면, 텍스트 영역의 hidden state가 **텍스트의 의미를 더 풍부하게 인코딩**하도록 강제된다. 이는 일종의 보조 손실(auxiliary loss)로, 텍스트 이해 능력을 강화하여 간접적으로 이미지 생성 품질을 높인다. 실제로 OpenAI는 텍스트 로스와 이미지 로스에 서로 다른 가중치를 적용했다.

위 수식을 단계별로 분해하면:

- **첫째 항** $-\sum_i \log p_\theta(t_i|t_{<i})$: 텍스트 내부의 자기회귀 예측. "yellow"을 보고 "bunch"를 예측, "yellow bunch"를 보고 "of"를 예측하는 식이다. 이 항은 Transformer가 텍스트의 문맥을 잘 이해하도록 만든다.
- **둘째 항** $-\sum_j \log p_\theta(s_j|t_{1:M}, s_{<j})$: 이미지 토큰의 자기회귀 예측. 전체 텍스트 $t_{1:M}$과 이전 이미지 토큰 $s_{<j}$를 조건으로 다음 이미지 토큰 s_j를 예측한다. 이것이 DALL-E의 핵심 학습 목표이다.
- 두 항 모두 **Cross-Entropy Loss**이며, 합산하여 하나의 backward pass로 최적화한다.

추론 (생성) 과정

새로운 프롬프트 "A fresh red apple"이 주어지면:

```
1. 텍스트 토큰화: [a, fresh, red, apple] → vocabulary 인덱스로 변환

2. 첫 번째 이미지 토큰 생성:
   입력: [a, fresh, red, apple]
   → 트랜스포머 디코더 → 소프트맥스 (8,192차원)
   → 샘플링 (argmax 또는 top-k) → y₁*

3. 두 번째 이미지 토큰 생성:
   입력: [a, fresh, red, apple, y₁*]
   → 트랜스포머 디코더 → 소프트맥스
   → 샘플링 → y₂*

4. 반복... 1,024개 토큰 또는 <end_of_image>까지

5. 생성된 이미지 토큰 [y₁*, y₂*, ..., y₁₀₂₄*]
   → 코드북에서 벡터 복원
   → dVAE 디코더 → 최종 이미지
```

> **첫 번째 이미지 토큰**은 텍스트 조건에 의해서만 결정. 이후 토큰은 텍스트 + 이전 이미지 토큰을 모두 어텐션.

같은 프롬프트에서 다양한 이미지가 나오는 이유

소프트맥스 분포로부터 **확률적 샘플링(stochastic sampling)**을 하기 때문. 확률이 높은 여러 코드 중 매번 다른 것이 선택될 수 있다.

이를 더 구체적으로 이해하자: 첫 번째 이미지 토큰 s_1을 생성할 때, 소프트맥스 출력이 예를 들어 코드 #3201에 15%, 코드 #5887에 12%, 코드 #1044에 8%의 확률을 부여할 수 있다. 샘플링할 때마다 다른 코드가 선택되고, 이 차이가 **눈덩이처럼 누적**된다. s_1이 달라지면 s_2의 조건이 달라지고, 그러면 s_2도 달라지고, 이것이 1,024개 토큰에 걸쳐 증폭되면서 최종적으로 완전히 다른 이미지가 생성된다. **Temperature** 파라미터로 이 다양성을 조절한다: temperature가 높으면(> 1) 확률 분포가 평탄해져서 더 다양하지만 품질이 낮아지고, 낮으면(< 1) 분포가 뾰족해져서 품질은 높지만 다양성이 줄어든다. DALL-E는 하나의 프롬프트에서 여러 이미지를 생성한 뒤, CLIP 모델로 텍스트와 가장 잘 맞는 이미지를 **재순위화(reranking)**하는 전략을 사용했다.

PyTorch 구현: Transformer Decoder for Autoregressive Generation

아래는 DALL-E Stage 2의 핵심인 자기회귀 트랜스포머 디코더의 간소화된 구현이다:

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class DALLETransformerDecoder(nn.Module):
    """
    DALL-E Stage 2: 텍스트 + 이미지 토큰의 자기회귀 트랜스포머 디코더
    텍스트 토큰과 이미지 토큰을 하나의 시퀀스로 연결하여 자기회귀 학습
    """
    def __init__(
        self,
        text_vocab_size=30000,     # 텍스트 vocabulary 크기 (BPE)
        image_vocab_size=8192,     # 이미지 vocabulary 크기 (dVAE 코드
        max_text_len=256,          # 최대 텍스트 시퀀스 길이
        max_image_len=1024,        # 이미지 토큰 수 (32x32)
        d_model=512,               # 트랜스포머 hidden 차원
        n_heads=8,                 # Multi-Head Attention 헤드 수
        n_layers=6,                # 트랜스포머 레이어 수
        dropout=0.1,
    ):
        super().__init__()
        self.max_text_len = max_text_len
        self.max_image_len = max_image_len
        self.d_model = d_model

        # 텍스트 토큰 임베딩 (별도의 vocabulary 공간)
        self.text_embedding = nn.Embedding(text_vocab_size, d_model)

        # 이미지 토큰 임베딩 (별도의 vocabulary 공간)
        self.image_embedding = nn.Embedding(image_vocab_size, d_mode

        # 위치 임베딩: 전체 시퀀스 길이 (텍스트 + 이미지)
        self.position_embedding = nn.Embedding(
            max_text_len + max_image_len, d_model
        )

        # 트랜스포머 디코더 레이어 스택
        decoder_layer = nn.TransformerDecoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=d_model * 4,
            dropout=dropout,
            batch_first=True,
        )
        self.transformer = nn.TransformerDecoder(
            decoder_layer, num_layers=n_layers
        )

        # 이미지 토큰 예측 헤드: d_model → 8,192 클래스
        self.image_head = nn.Linear(d_model, image_vocab_size)
```

```python
        # 텍스트 토큰 예측 헤드: d_model → 30,000 클래스
        self.text_head = nn.Linear(d_model, text_vocab_size)

    def _generate_causal_mask(self, seq_len, device):
        """
        Causal Mask 생성: 미래 토큰에 대한 어텐션을 차단
        상삼각 행렬을 -inf로 설정하여 소프트맥스 후 어텐션 가중치 = 0
        """
        mask = torch.triu(
            torch.ones(seq_len, seq_len, device=device), diagonal=1
        ).bool()
        return mask  # True 위치가 마스킹됨

    def forward(self, text_tokens, image_tokens):
        """
        훈련용 forward pass (Teacher Forcing)

        Args:
            text_tokens: 텍스트 토큰 인덱스, shape = (B, M)
            image_tokens: 이미지 토큰 인덱스, shape = (B, 1024)
        Returns:
            text_logits: 텍스트 예측 로짓, shape = (B, M, text_vocab_
            image_logits: 이미지 예측 로짓, shape = (B, 1024, image_v
        """
        B = text_tokens.size(0)
        M = text_tokens.size(1)
        N = image_tokens.size(1)
        device = text_tokens.device

        # 각각의 임베딩 레이어로 변환
        text_emb = self.text_embedding(text_tokens)    # (B, M, d_mo
        image_emb = self.image_embedding(image_tokens)  # (B, N, d_r

        # 텍스트 + 이미지 토큰 연결 (concatenation)
        combined_emb = torch.cat([text_emb, image_emb], dim=1)  # (I

        # 위치 임베딩 추가
        positions = torch.arange(M + N, device=device).unsqueeze(0)
        combined_emb = combined_emb + self.position_embedding(posit:

        # Causal Mask 생성 및 적용
        causal_mask = self._generate_causal_mask(M + N, device)

        # 트랜스포머 디코더 실행 (디코더-only 구조이므로 memory는 자기
        # nn.TransformerDecoder를 self-attention으로 사용
        output = self.transformer(
            combined_emb, combined_emb, tgt_mask=causal_mask,
            memory_mask=causal_mask
        )  # (B, M+N, d_model)

        # 텍스트 영역과 이미지 영역 분리
        text_output = output[:, :M, :]      # (B, M, d_model)
```

```
        text_output = output[:, :M, :]      # (B, M, d_model)
        image_output = output[:, M:, :]     # (B, N, d_model)

        # 각각의 예측 헤드 적용
        text_logits = self.text_head(text_output)     # (B, M, text_
        image_logits = self.image_head(image_output)  # (B, N, imag

        return text_logits, image_logits

    def compute_loss(self, text_tokens, image_tokens):
        """
        DALL-E 훈련 손실 계산:
        L = -sum_i log p(t_i | t_{<i}) - sum_j log p(s_j | t_{1:M},

        텍스트 영역과 이미지 영역 모두에서 Cross-Entropy 계산
        """
        text_logits, image_logits = self.forward(text_tokens, image_

        # 텍스트 다음 토큰 예측 손실: t_i 위치의 출력으로 t_{i+1} 예측
        # text_logits[:, :-1]는 마지막 토큰 제외, target은 첫 토큰 제
        text_loss = F.cross_entropy(
            text_logits[:, :-1].reshape(-1, text_logits.size(-1)),
            text_tokens[:, 1:].reshape(-1),
            ignore_index=-1,  # 패딩 토큰 무시
        )

        # 이미지 다음 토큰 예측 손실: s_j 위치의 출력으로 s_{j+1} 예측
        image_loss = F.cross_entropy(
            image_logits[:, :-1].reshape(-1, image_logits.size(-1))
            image_tokens[:, 1:].reshape(-1),
        )

        # 전체 손실 = 텍스트 손실 + 이미지 손실
        total_loss = text_loss + image_loss
        return total_loss, text_loss.item(), image_loss.item()

    @torch.no_grad()
    def generate(self, text_tokens, top_k=50, temperature=1.0):
        """
        자기회귀 이미지 생성 (추론 시)

        Args:
            text_tokens: 텍스트 토큰 인덱스, shape = (B, M)
            top_k: top-k 샘플링에서의 k 값
            temperature: 소프트맥스 온도 (높을수록 다양, 낮을수록 집중)
        Returns:
            generated_indices: 생성된 이미지 토큰, shape = (B, 1024)
        """
        B = text_tokens.size(0)
        device = text_tokens.device

        # 생성된 이미지 토큰을 저장할 텐서
```

```python
        generated = torch.zeros(B, 0, dtype=torch.long, device=devi

        for step in range(self.max_image_len):
            # 현재까지 생성된 시퀀스로 다음 토큰 예측
            if generated.size(1) == 0:
                # 첫 번째 이미지 토큰: 텍스트만으로 예측
                dummy_img = torch.zeros(B, 1, dtype=torch.long, dev
                _, img_logits = self.forward(text_tokens, dummy_img
                next_logits = img_logits[:, 0, :]  # 첫 위치의 로짓
            else:
                _, img_logits = self.forward(text_tokens, generated
                next_logits = img_logits[:, -1, :]  # 마지막 위치의 

            # Temperature 적용
            next_logits = next_logits / temperature

            # Top-k 샘플링: 상위 k개만 남기고 나머지는 -inf로 마스킹
            if top_k > 0:
                values, _ = torch.topk(next_logits, top_k, dim=-1)
                min_values = values[:, -1].unsqueeze(-1)
                next_logits[next_logits < min_values] = float('-inf

            # 확률적 샘플링 (같은 프롬프트에서 다양한 이미지를 생성하는
            probs = F.softmax(next_logits, dim=-1)
            next_token = torch.multinomial(probs, num_samples=1)  #

            generated = torch.cat([generated, next_token], dim=1)

        return generated  # (B, 1024)


# === 사용 예시 ===
if __name__ == "__main__":
    model = DALLETransformerDecoder(
        text_vocab_size=30000,
        image_vocab_size=8192,
        d_model=512,
        n_heads=8,
        n_layers=6,
    )

    # 더미 텍스트 토큰 (예: "a fresh red apple"의 BPE 인코딩)
    text_tokens = torch.randint(0, 30000, (2, 8))   # batch=2, 텍스
    image_tokens = torch.randint(0, 8192, (2, 1024)) # batch=2, 이미

    # 훈련 시: 손실 계산
    total_loss, text_loss, image_loss = model.compute_loss(text_toke
    print(f"전체 손실: {total_loss.item():.4f}")
    print(f"텍스트 손실: {text_loss:.4f}, 이미지 손실: {image_loss:.4

    # 추론 시: 자기회귀 이미지 생성
    # (실제로는 1024 스텝이 소요되므로 시간이 걸림)
```

```
# generated = model.generate(text_tokens, top_k=50, temperature=
# print(f"생성된 토큰 shape: {generated.shape}")  # (2, 1024)
```

1.4 DALL-E의 생성 결과

- 비현실적 이미지: "아코디언으로 만들어진 돌고래" 등 실제로 존재하지 않는 것
- 텍스트 렌더링: "backprop"이라고 쓰인 네온 사인 (단, 스펠링 오류 빈번)
- Zero-shot 생성: 훈련 때 없던 조합도 생성 가능

> 💡 **왜 Zero-shot 생성이 가능한가?** "아코디언으로 만들어진 돌고래"라는 이미지는 훈련 데이터에 존재하지 않을 것이다. 그런데 DALL-E가 이를 생성할 수 있는 이유는 **조합적 일반화(compositional generalization)** 때문이다. 모델은 "아코디언"의 시각적 특징, "돌고래"의 형태, "~로 만들어진"이라는 관계를 각각 학습했고, 이들을 **새로운 조합**으로 합성할 수 있다. 이는 언어 모델이 "보라색 코끼리가 스케이트를 탄다"같은 문장을 본 적이 없어도 문법적으로 올바른 문장을 생성할 수 있는 것과 같은 원리이다. 2억 5천만 개의 텍스트-이미지 쌍에서 학습한 광범위한 시각적 개념과 텍스트-이미지 대응 관계가 이러한 조합적 창의성의 기반이 된다.

> ⚠️ **DALL-E의 한계**: 복잡한 공간 관계("A위에 B, B 옆에 C" 등)나 정확한 텍스트 렌더링에서는 성능이 제한적이었다. 이러한 한계는 이후 DALL-E 2, DALL-E 3에서 Diffusion 기반 구조와 개선된 텍스트 인코더(CLIP)를 통해 크게 개선되었다.

1.5 CogView / Crayon: 인코더-디코더 구조

> 📄 **논문**: Ding et al., "*CogView: Mastering Text-to-Image Generation via Transformers*", NeurIPS 2021
>
> CogView: 중국 연구팀 발표 (2021.07), 원래 이름 "DALL-E mini" → OpenAI 요청으로 **Crayon**으로 개명

DALL-E와의 차이

	DALL-E	**CogView / Crayon**
이미지 토크나이저	dVAE	**VQ-GAN**
트랜스포머 구조	**디코더 only**	**인코더-디코더**
텍스트 처리	이미지 토큰과 동등하게 concat	인코더에서 **별도 처리** 후 디코더로 전달

인코더-디코더 구조의 장점

DALL-E(디코더 only): 텍스트 토큰이 이미지 토큰과 동등한 레벨로 시퀀스에 포함 → 텍스트에 대한 Self-Attention이 점진적으로 이루어짐

Crayon(인코더-디코더): 텍스트를 **인코더가 먼저 완전히 처리** → 정제된 표현이 디코더에 Key/Value로 전달

```
[인코더]
텍스트 토큰 → [Transformer Encoder xN] → 히든 표현 H = [h₁, h₂, h₃, h
                                                          ↓ (Key,
[디코더]
이미지 토큰 → [Transformer Decoder xN] → 다음 이미지 토큰 예측
              (Self-Attention + Cross-Attention to H)
```

- 인코더의 히든 표현은 **Key와 Value로만** 사용 (출력 없음)
- 디코더는 이미지 토큰의 Self-Attention + 인코더 출력에 대한 **Cross-Attention**
- 텍스트 전체의 **요약된 의미**에 직접 접근 가능 → 성능 향상

> 💡 **왜 인코더-디코더 구조가 더 나은가? -- 구체적 예시로 이해하기** "A red cat sitting on a blue chair"라는 텍스트를 생각하자. 디코더-only 구조(DALL-E)에서 이미지 토큰 s_{500}을 생성할 때, 이 토큰은 텍스트의 모든 토큰에 self-attention을 수행한다. 하지만 텍스트 토큰 "red"와 "cat"의 관계, "blue"와 "chair"의 관계는 **텍스트 내부의 문맥**에서 먼저 이해되어야 한다. 디코더-only에서는 이 문맥 이해가 이미지 토큰과의 attention과 혼재되어 비효율적이다. 반면 인코더-디코더 구조에서는 인코더가 **텍스트만을 위한 양방향(bidirectional) attention**으로 "red→cat", "blue→chair"라는 속성-대상 관계를 먼저 완전히 파악한다. 디코더는 이미 **정제된 의미 표현**에 cross-attention하므로, "빨간색은 고양이에, 파란색은 의자에"라는 바인딩을 더 정확하게 반영할 수 있다.

> **핵심 차이**: 날것의 텍스트 토큰에 어텐션 vs 인코더가 정제한 표현에 어텐션. 후자가 더 효과적.

이후 모델들(DALL-E 2, Imagen 등)은 모두 텍스트 인코더(CLIP 등)를 별도로 사용하는 방향으로 발전. 이는 CogView의 인코더-디코더 아이디어가 옳았음을 보여준다.

---

2. Score-Based Generative Models

> 📄 **핵심 참고**: Stefano Ermon & Yang Song의 ICLR 튜토리얼
>
> 📄 **논문**: Song & Ermon, "*Generative Modeling by Estimating Gradients of the Data Distribution*", NeurIPS 2019
>
> 현재 생성 모델의 **주류**를 이루는 방식의 이론적 근간

2.1 기존 생성 모델의 한계

> 💡 **왜 지금까지의 접근법으로는 부족한가?** DALL-E까지 배운 생성 모델들은 모두 어떤 형태로든 $p(x)$를 다뤄야 했고, 각각 고유한 제약이 있었다. VAE는 ELBO라는 하한만 최적화하므로 이론적으로 최적이 아니고, GAN은 훈련이 불안정하며 모드

> 붕괴(mode collapse)에 취약하고, Normalizing Flow는 아키텍처에 강한 제약(가역성, 야코비안 계산 가능성)이 필요하고, 자기회귀 모델은 생성 속도가 느리다. "이 모든 문제의 근원이 $p(x)$를 직접 모델링하려는 데 있지는 않은가?"라는 근본적 질문에서 Score-Based Model이 탄생했다.

지금까지 배운 생성 모델들은 모두 **확률 분포 $p(x)$를 직접 모델링**했다:

모델	$p(x)$ **추정 방식**
VAE	ELBO 최대화로 근사
VQ-VAE	이산적 사전 분포 학습
PixelCNN/RNN	$p(x) = \prod_i p(x_i \mid x_{<i})$ 자기회귀 분해
Transformer (VQ-GAN, DALL-E)	코드북 토큰의 자기회귀 확률
GAN	암묵적 밀도 추정 (직접 $p(x)$를 배우지 않음)
Normalizing Flow	변수 변환으로 정확한 $p(x)$ 계산

확률 분포 $p(x)$를 직접 다루는 것의 어려움:

$$\int p(x)\, dx = 1 \quad (\text{정규화 조건})$$

모델을 변형할 때마다 이 적분이 1이 되도록 **정규화 상수(normalizing constant)**를 계산해야 하는데, 이것이 매우 어렵다.

> ⚠️ **왜 정규화가 어려운가**: 고차원 공간(예: $256 \times 256 \times 3 = 196{,}608$ 차원의 이미지)에서의 적분은 해석적으로 풀 수 없고, 수치 적분도 차원의 저주(curse of dimensionality)로 인해 사실상 불가능하다. 이것이 VAE에서 ELBO라는 하한(lower bound)을 쓸 수밖에 없는 근본 이유이기도 하다.

정규화 상수 Z가 왜 계산 불가능한지 구체적으로 이해하자. 에너지 기반 모델(Energy-Based Model, EBM)에서 확률 분포를 다음과 같이 정의한다고 하자:

$$p_\theta(x) = \frac{1}{Z(\theta)} \exp(-E_\theta(x)), \quad Z(\theta) = \int \exp(-E_\theta(x))\, dx$$

여기서 $E_\theta(x)$는 뉴럴넷으로 자유롭게 설계할 수 있는 에너지 함수이다. 문제는 $Z(\theta)$이다. 이미지 x가 $d = 196{,}608$ 차원이라면, 이 적분은 196,608차원 공간 전체에 걸친 적분이다. 수치 적분을 시도한다면, 각 차원을 10개 구간으로만 나눠도 $10^{196{,}608}$개의 점을 평가해야 한다 -- 이는 관측 가능한 우주의 원자 수($\approx 10^{80}$)보다 상상할 수 없을 만큼 크다. Monte Carlo 추정도 이 차원에서는 분산이 폭발적으로 커져 실용적이지 않다. 이것이 바로 **차원의 저주(curse of dimensionality)**이며, $p(x)$를 직접 다루는 모든 모델이 겪는 근본적 병목이다.

각 모델이 이 문제를 어떻게 우회했는지 정리하면:

- **VAE**: $p(x)$를 직접 계산하지 않고, 하한(ELBO)만 최적화. 하한이므로 최적은 아님
- **GAN**: $p(x)$를 명시적으로 모델링하지 않고, 판별자와의 게임으로 암묵적 학습. 하지만 훈련 불안정
- **Normalizing Flow**: 가역 변환의 야코비안으로 정확한 $p(x)$ 계산. 하지만 아키텍처 제약이 심함
- **자기회귀 모델**: 조건부 확률의 곱으로 분해하여 Z 문제를 회피. 하지만 생성이 순차적이라 느림
- **Score-Based Model**: $p(x)$ **자체를 버리고 기울기만 학습** → 이것이 이번 섹션의 핵심

2.2 핵심 발상의 전환: 확률 대신 기울기를 학습

> **$p(x)$를 직접 찾지 말고, $p(x)$의 기울기(gradient)를 찾자**

> 💡 **이 발상이 왜 혁명적인가?** 지도(확률 분포 $p(x)$)를 완벽하게 그리는 것은 불가능에 가깝다. 하지만 "어느 방향으로 가면 높은 곳(데이터가 많은 곳)인가?"라는 **방향 정보(기울기)**만 있으면, 임의의 위치에서 출발해서 높은 곳에 도달할 수 있다. 산의 전체 지형도를 그리지 못하더라도, 모든 위치에서의 "오르막 방향"만 알면 정상에 도달할 수 있는 것과 같다. 이 관점의 전환이 정규화 상수 Z라는 근본적 난제를 완전히 우회할 수 있게 해준다.

스코어 함수(Score Function) 정의

$$\boxed{s(x) = \nabla_x \log p(x)}$$

- ∇_x: x**에 대한** 미분 (파라미터 θ가 아님!)
- $\log p(x)$: 로그 확률 밀도
- 출력: x와 같은 차원의 벡터 (방향 + 크기)

> ⚠️ **혼동 주의**: 머신러닝에서 일반적으로 사용하는 그래디언트 $\nabla_\theta \mathcal{L}$은 **파라미터 θ**에 대한 미분이지만, 스코어 함수에서의 ∇_x는 **입력 데이터 x**에 대한 미분이다. 이 구분이 매우 중요하다.

스코어의 의미

스코어는 "현재 위치 x에서 **확률이 높아지는 방향**"을 가리키는 벡터:

- 데이터가 많이 분포하는 곳(모드) → 스코어가 그 방향을 가리킴
- 모드의 정확한 중심 → 기울기 = 0 (극대점)
- 모드에서 벗어나면 → 다시 모드 방향으로 끌어당기는 벡터

중력 비유

우주 공간에서의 중력에 대응시키면 직관적이다:

개념	확률 분포	중력 비유

데이터 포인트 x	우주 공간의 한 점	임의의 우주 위치
$p(x)$가 높은 곳	데이터가 밀집한 영역	행성/별 (목성, 토성 등)
$p(x) \approx 0$	데이터가 없는 영역	텅 빈 우주 공간
스코어 $\nabla_x \log p(x)$	확률 증가 방향	**중력** (행성 방향으로 끌어당기는 힘)
모드 (최빈값)	확률 밀도의 극대점	행성의 중심 (중력 = 0)

행성 표면: 중력이 가장 강함 (스코어의 절댓값이 큼) 행성 중심: 중력 = 0 (스코어 = 0, 극대점) 행성에서 멀어지면: 다시 행성 방향으로 중력 작용

벡터 필드(Vector Field)

모든 위치 x에서의 스코어를 모아 놓으면 **벡터 필드**가 된다. 화살표의 방향은 확률이 높아지는 방향, 길이는 기울기의 크기를 나타낸다.

2.3 스코어 계산 예시: 가우시안 분포

가우시안 분포 $p(x) = \mathcal{N}(x; \mu, \sigma^2)$에 대한 스코어를 직접 계산해보자. 이 예시를 통해 스코어의 의미를 구체적으로 체감할 수 있다.

Step 1: 가우시안 확률 밀도 함수를 쓴다

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

여기서 $\frac{1}{\sqrt{2\pi}\sigma}$가 바로 정규화 상수이다. 이 값이 있어야 $\int p(x)dx = 1$이 성립한다.

Step 2: 로그를 취한다. 로그는 곱셈을 덧셈으로, 지수를 일반 곱으로 바꿔준다

$$\log p(x) = \underbrace{-\frac{1}{2}\log(2\pi\sigma^2)}_{\text{상수 (x와 무관)}} - \frac{(x-\mu)^2}{2\sigma^2}$$

핵심 관찰: 첫째 항은 x를 포함하지 않는 상수이다. 이것이 정규화 상수에 해당하는 부분이다.

Step 3: x에 대해 미분한다. 상수를 미분하면 0이 되므로 첫째 항이 사라진다

$$s(x) = \nabla_x \log p(x) = 0 - \frac{2(x-\mu)}{2\sigma^2} = -\frac{x-\mu}{\sigma^2}$$

정규화 상수에 해당하는 항이 미분 과정에서 **자연스럽게 소멸**했다. 이것이 스코어의 핵심 장점을 보여주는 가장 간단한 예시이다.

> 결과 해석: $x > \mu$이면 스코어는 음수 (왼쪽으로, 즉 μ 방향으로 이동), $x < \mu$이면 양수 (오른쪽으로, μ 방향으로 이동). **항상 평균 μ를 향해** 끌어당긴다.

형태: 거의 직선. 확률 분포보다 훨씬 단순하다.

다변량(multivariate) 가우시안으로 확장하면: $p(x) = \mathcal{N}(x; \mu, \Sigma)$일 때, 스코어는 $s(x) = -\Sigma^{-1}(x - \mu)$이다. 벡터 형태이므로 각 차원마다 평균 방향으로 끌어당기는 힘의 방향과 크기가 정의된다. 공분산 Σ가 작은 차원(분산이 좁은 축)에서는 스코어의 크기가 크고(강하게 끌어당기고), 큰 차원에서는 스코어가 작다(부드럽게 끌어당긴다).

2.4 스코어의 장점: 정규화 불필요

확률 분포 $p(x)$는 면적(적분)이 항상 1이어야 한다. 이 정규화 상수를 구하는 것이 매우 어렵다.

그러나 스코어에서는 이 문제가 사라진다. 단계별로 유도하자:

Step 1: 임의의 확률 분포를 정규화되지 않은 부분과 정규화 상수로 분리한다

$$p(x) = \frac{\tilde{p}(x)}{Z}, \quad Z = \int \tilde{p}(x)\, dx$$

여기서 $\tilde{p}(x)$는 뉴럴넷이 자유롭게 출력할 수 있는 양수 함수이고, Z는 이를 적분하여 1로 만드는 상수이다.

Step 2: 로그를 취한다

$$\log p(x) = \log \tilde{p}(x) - \log Z$$

Step 3: x에 대해 미분한다. Z는 x에 대한 적분의 결과이므로 x를 포함하지 않는 **상수**이다

$$\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x) - \underbrace{\nabla_x \log Z}_{=0} = \nabla_x \log \tilde{p}(x)$$

Z는 x에 무관한 상수이므로 미분하면 0. 따라서 **정규화 상수를 전혀 신경 쓸 필요가 없다**.

> 💡 **이것이 실용적으로 의미하는 바:** 뉴럴넷 $f_\theta(x)$를 자유롭게 설계하고, $\tilde{p}_\theta(x) = \exp(-f_\theta(x))$로 놓으면, $\nabla_x \log \tilde{p}_\theta(x) = -\nabla_x f_\theta(x)$이다. 이 그래디언트는 PyTorch의 autograd로 즉시 계산할 수 있다. 정규화 상수 $Z(\theta) = \int \exp(-f_\theta(x)) dx$를 계산할 필요가 전혀 없다. 에너지 기반 모델(EBM)에서는 Z를 근사하기 위해 비용이 큰 MCMC 샘플링이 필요했지만, Score-Based Model은 이 문제를 완전히 제거한다. 이것이 Score-Based Model의 **가장 강력한 이론적 장점**이다.

2.5 스코어 매칭 (Score Matching)

> 💡 **왜 스코어 "매칭"이 필요한가?** 스코어 함수의 이론적 장점은 이해했다. 하지만 실제로 스코어를 학습하려면, 뉴럴넷의 출력 $s_\theta(x)$가 진짜 스코어 $\nabla_x \log p_{\text{data}}(x)$와 **일치(match)**하도록 훈련해야 한다. 문제는 진짜 스코어 $\nabla_x \log p_{\text{data}}(x)$를 우리가 모른다는 것이다 -- $p_{\text{data}}(x)$ 자체를 모르니까! 이 역설적 상황을 해결하는 것이 **스코어 매칭**이론의 핵심이다. Aapo Hyvarinen이 2005년에 발표한 이 이론은, 놀랍게도 진짜 스코어를 모르는 상태에서도 동등한 목적 함수를 유도할 수 있음을 보여준다.

목표

뉴럴넷 $s_\theta(x)$가 실제 스코어 $\nabla_x \log p_{\text{data}}(x)$를 학습하도록 훈련:

$$s_\theta(x) \approx \nabla_x \log p_{\text{data}}(x)$$

- 뉴럴넷 구조: **U-Net** (입력 차원 = 출력 차원이어야 하므로)
- 입력: 이미지 x
- 출력: 같은 차원의 스코어 벡터

> 💡 **왜 U-Net인가?** 스코어 함수는 입력 x와 **동일한 차원**의 벡터를 출력해야 한다 (각 픽셀 위치에서의 기울기 방향). 이는 입력과 출력의 해상도가 같아야 하므로, semantic segmentation에서 검증된 **U-Net** 구조가 자연스러운 선택이다. U-Net의 skip connection은 저수준 특징(경계선, 텍스처)과 고수준 특징(객체 구조)을 모두 결합하므로, 픽셀 단위의 정밀한 스코어 추정에 적합하다.

훈련 목표 (이상적)

두 스코어의 거리를 최소화:

$$\mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}\left[\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|_2^2\right]$$

이것은 **Fisher Divergence**에 해당한다. 직관적으로, 모든 데이터 포인트 x에서 뉴럴넷이 예측한 스코어와 진짜 스코어 사이의 L2 거리를 평균한 것이다. 이 값이 0이면 완벽한 스코어 학습이 된다.

문제: $\nabla_x \log p_{\text{data}}(x)$를 모른다! ($p_{\text{data}}$를 모르므로)

이 상황은 "정답을 모르는데 정답과의 거리를 최소화해야 한다"는 역설처럼 보인다. 하지만 수학이 이 문제를 해결해준다.

부분 적분을 통한 변환

부분 적분(integration by parts)을 적용하면, 정답 스코어를 모르는 상태에서도 동등한 목적 함수를 유도할 수 있다. **유도 과정을 단계별로 따라가보자:**

Step 1: Fisher Divergence를 전개한다

$$\mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}_x\left[\|s_\theta(x)\|^2 - 2s_\theta(x)^\top \nabla_x \log p_{\text{data}}(x) + \|\nabla_x \log p_{\text{data}}(x)\|^2\right]$$

셋째 항 $\|\nabla_x \log p_{\text{data}}(x)\|^2$는 θ에 무관하므로 최적화에 영향을 주지 않는다. 따라서 무시할 수 있다.

Step 2: 핵심은 둘째 항 $\mathbb{E}_x[s_\theta(x)^\top \nabla_x \log p_{\text{data}}(x)]$를 변환하는 것이다. $\nabla_x \log p_{\text{data}}(x) = \frac{\nabla_x p_{\text{data}}(x)}{p_{\text{data}}(x)}$를 대입하면:

$$\mathbb{E}_{x\sim p_{\text{data}}}[s_\theta(x)^\top \nabla_x \log p_{\text{data}}(x)] = \int s_\theta(x)^\top \nabla_x p_{\text{data}}(x)\, dx$$

Step 3: 부분 적분을 적용한다. $\int f \cdot g' dx = [fg]_{-\infty}^{\infty} - \int f' \cdot g\, dx$에서 $p_{\text{data}}(x)$가 경계에서 0으로 수렴한다는 조건 하에:

$$\int s_\theta(x)^\top \nabla_x p_{\text{data}}(x)\, dx = -\int \text{tr}(\nabla_x s_\theta(x))\, p_{\text{data}}(x)\, dx = -\mathbb{E}_x[\text{tr}(\nabla_x s_\theta(x))]$$

Step 4: 이를 대입하면 최종 목적 함수가 된다:

$$\mathcal{L}(\theta) = \mathbb{E}_{x\sim p_{\text{data}}}\left[\text{tr}(\nabla_x s_\theta(x)) + \frac{1}{2}\|s_\theta(x)\|_2^2\right]$$

핵심: $\nabla_x \log p_{\text{data}}(x)$가 완전히 사라졌다! 이 목적 함수는 오직 $s_\theta(x)$와 그 야코비안만 포함한다.

여기서 $\text{tr}(\nabla_x s_\theta(x))$는 s_θ의 **야코비안(Jacobian)의 트레이스(trace)**:

$$\nabla_x s_\theta(x) = \begin{pmatrix} \frac{\partial s_{\theta,1}}{\partial x_1} & \cdots & \frac{\partial s_{\theta,1}}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial s_{\theta,n}}{\partial x_1} & \cdots & \frac{\partial s_{\theta,n}}{\partial x_n} \end{pmatrix}$$

$$\text{tr}(\nabla_x s_\theta) = \sum_{i=1}^{n} \frac{\partial s_{\theta,i}}{\partial x_i}$$

$p_{\text{data}}(x)$가 사라졌으므로, 데이터 샘플만으로 계산 가능:

$$\mathcal{L}(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left[\text{tr}(\nabla_x s_\theta(x_i)) + \frac{1}{2}\|s_\theta(x_i)\|_2^2\right]$$

계산량 문제와 Sliced Score Matching

야코비안의 트레이스 계산은 x의 차원이 클 때 **계산량이 매우 크다** (이미지는 수만 차원).

> 💡 **왜 야코비안 트레이스가 비싼가?** $\text{tr}(\nabla_x s_\theta(x)) = \sum_{i=1}^{n} \frac{\partial s_{\theta,i}}{\partial x_i}$를 계산하려면, 스코어의 각 출력 차원 $s_{\theta,i}$에 대해 x_i로의 편미분을 구해야 한다. 이미지가 $n = 196{,}608$ 차원이면, **196,608번의 역전파(backpropagation)**가 필요하다. 한 번의 역전파로는 하나의 스칼라 출력에 대한 그래디언트만 얻을 수 있기 때문이다. 이는 단 하나의 훈련 샘플에 대해서도 감당하기 어려운 계산량이다.

해결책은 두 가지 방향이 있다:

방법 1: Sliced Score Matching -- 전체 벡터를 비교하는 대신, **랜덤한 방향으로 프로젝션**한 스칼라 값만 비교:

1. 랜덤 방향 벡터 v를 샘플링 (Rademacher 분포: 각 원소가 +1 또는 -1)
2. 스코어를 v 방향으로 프로젝션: $v^\top s_\theta(x)$ (스칼라)
3. 이 스칼라의 x에 대한 그래디언트를 구함: $\nabla_x(v^\top s_\theta(x))$ -- 이것은 **한 번의 역전파**로 계산 가능
4. 프로젝션된 야코비안-벡터 곱: $v^\top \nabla_x s_\theta(x) \cdot v$를 로스에 사용

이를 여러 랜덤 방향에 대해 반복하면, 원래의 스코어 매칭과 동등한 결과를 얻으면서 **계산량은 대폭 감소**.

방법 2: Denoising Score Matching (DSM) -- 이것이 실전에서 가장 널리 사용되는 방법이며, 야코비안을 아예 계산하지 않는다.

> 💡 **왜 Denoising Score Matching이 작동하는가? (핵심 증명)** 깨끗한 데이터 x에 가우시안 노이즈를 추가하여 $\tilde{x} = x + \sigma n$ ($n \sim \mathcal{N}(0, I)$)을 만들면, 조건부 분포 $q_\sigma(\tilde{x}|x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$의 스코어는 **해석적으로 알려져 있다**:
>
> $$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{n}{\sigma}$$
>
> Vincent(2011)의 핵심 결과: 노이즈가 추가된 데이터 분포 $q_\sigma(\tilde{x}) = \int q_\sigma(\tilde{x}|x) p_{\text{data}}(x) dx$에 대해, 다음 두 목적 함수가 **동등**하다:
>
> $$\mathbb{E}_{\tilde{x} \sim q_\sigma}\left[\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x})\|^2\right] = \mathbb{E}_{x \sim p_{\text{data}}, n \sim \mathcal{N}}\left[\|s_\theta(x + \sigma n) + \frac{n}{\sigma}\|^2\right] + C$$
>
> 좌변은 계산 불가능한 진짜 스코어와의 매칭이고, 우변은 깨끗한 데이터와 노이즈만으로 계산 가능한 형태이다. 상수 C는 θ에 무관하므로 최적화에 영향을 주지 않는다. 따라서 **야코비안 계산 없이, 단순한 MSE 손실만으로** 스코어를 학습할 수 있다. 이것이 실제 Score-Based Model과 DDPM에서 사용되는 학습 방법이다.

PyTorch 구현: Score Matching Loss

아래는 스코어 매칭의 세 가지 변형 (기본, Denoising, Sliced)을 PyTorch로 구현한 코드이다:

```python
import torch
import torch.nn as nn
import torch.autograd as autograd

class SimpleScoreNetwork(nn.Module):
    """
    간단한 스코어 네트워크: 입력 x와 같은 차원의 스코어 벡터를 출력
    실제 이미지에는 U-Net을 사용하지만, 원리 이해를 위한 간소화 버전
    """
    def __init__(self, input_dim=2, hidden_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.SiLU(),  # Swish 활성화 함수
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, input_dim),  # 출력 차원 = 입력 차
        )

    def forward(self, x):
        """
        Args:
            x: 입력 데이터, shape = (B, D)
        Returns:
            score: 스코어 벡터 s_θ(x), shape = (B, D)
        """
        return self.net(x)


def explicit_score_matching_loss(score_net, x):
    """
    기본 스코어 매칭 손실 (야코비안 트레이스 포함):
    L(θ) = E_x [ tr(▽_x s_θ(x)) + 1/2 ||s_θ(x)||² ]

    주의: 야코비안 트레이스 계산이 비싸므로 고차원에서는 비실용적
    원리 학습용으로만 사용

    Args:
        score_net: 스코어 네트워크 s_θ
        x: 데이터 샘플, shape = (B, D), requires_grad=True 필요
    """
    x = x.requires_grad_(True)
    score = score_net(x)  # s_ϑ(x), shape = (B, D)

    # ||s_ϑ(x)||² 항: 스코어 벡터의 L2 norm 제곱
    score_norm = 0.5 * torch.sum(score ** 2, dim=-1)  # (B,)

    # tr(▽_x s_ϑ(x)) 항: 야코비안의 대각 원소 합
    # 각 차원 i에 대해 ∂s_ϑ,i / ∂x_i를 계산
    trace = torch.zeros(x.shape[0], device=x.device)
```

```
    for i in range(x.shape[1]):
        # score[:, i]를 x에 대해 미분 → 야코비안의 i번째 행
        grad_i = autograd.grad(
            outputs=score[:, i].sum(),
            inputs=x,
            create_graph=True,  # 2차 미분을 위해 계산 그래프 유지
        )[0]  # shape = (B, D)
        trace += grad_i[:, i]  # 대각 원소만 추출하여 합산

    # 전체 손실: tr(∇_x s_ϑ) + 1/2 ||s_ϑ||²
    loss = torch.mean(trace + score_norm)
    return loss


def denoising_score_matching_loss(score_net, x, sigma=0.1):
    """
    Denoising Score Matching (DSM) 손실:
    노이즈가 추가된 데이터에 대해 스코어를 학습하는 실용적 방법

    핵심 아이디어: x~ = x + σn (n ~ N(0,I)) 일 때,
    노이즈가 추가된 분포의 스코어는 알려져 있다:
    ∇_{x~} log p_σ(x~|x) = -(x~ - x) / σ²

    따라서 이 정답 스코어에 맞추면 된다:
    L = E_{x,n} [ ||s_θ(x~) - (-(x~ - x)/σ²)||² ]
      = E_{x,n} [ ||s_θ(x~) + n/σ||² ]

    Args:
        score_net: 스코어 네트워크 s_θ
        x: 깨끗한 데이터 샘플, shape = (B, D)
        sigma: 노이즈 수준
    """
    # 가우시안 노이즈 추가
    noise = torch.randn_like(x)       # n ~ N(0, I)
    x_noisy = x + sigma * noise        # x~ = x + σn

    # 스코어 네트워크 예측
    score = score_net(x_noisy)         # s_ϑ(x~), shape = (B, D)

    # 정답 스코어: -(x~ - x) / σ² = -n / σ
    target = -noise / sigma

    # MSE 손실: ||s_ϑ(x~) - target||²
    loss = 0.5 * torch.mean(torch.sum((score - target) ** 2, dim=-1
    return loss


def sliced_score_matching_loss(score_net, x, n_projections=4):
    """
    Sliced Score Matching (SSM) 손실:
    랜덤 프로젝션을 통해 야코비안 트레이스 계산을 우회하는 효율적 방법
```

```
    핵심: 전체 야코비안 대신, 랜덤 방향 v로 프로젝션한 스칼라로 계산
    L = E_v E_x [ v^T ▽_x s_θ(x) v + 1/2 (v^T s_θ(x))² ]

    Args:
        score_net: 스코어 네트워크 s_θ
        x: 데이터 샘플, shape = (B, D)
        n_projections: 랜덤 프로젝션 벡터 수 (많을수록 분산 감소)
    """
    x = x.requires_grad_(True)
    score = score_net(x)  # s_ϑ(x), shape = (B, D)

    total_loss = 0.0
    for _ in range(n_projections):
        # 랜덤 방향 벡터 v 샘플링 (Rademacher 분포: ±1 균등)
        v = torch.randint(0, 2, x.shape, device=x.device).float() *

        # v^T s_ϑ(x): 스코어를 v 방향으로 프로젝션 (스칼라)
        sv = torch.sum(score * v, dim=-1)  # (B,)

        # v^T ▽_x s_ϑ(x) v: 프로젝션된 야코비안-벡터 곱
        # ▽_x (v^T s_ϑ(x)) 를 autograd로 효율적 계산
        grad_sv = autograd.grad(
            outputs=sv.sum(),
            inputs=x,
            create_graph=True,
        )[0]  # shape = (B, D)
        jvp = torch.sum(grad_sv * v, dim=-1)  # v^T * ( ▽_x (v^T s_ϑ

        # 손실: v^T J v + 1/2 (v^T s)²
        total_loss += torch.mean(jvp + 0.5 * sv ** 2)

    return total_loss / n_projections


# === 사용 예시: 2D 가우시안 혼합 분포에서 스코어 학습 ===
if __name__ == "__main__":
    torch.manual_seed(42)

    # 2D 가우시안 혼합 데이터 생성 (모드 2개)
    n_samples = 1000
    mix = torch.randint(0, 2, (n_samples,))
    means = torch.tensor([[2.0, 2.0], [-2.0, -2.0]])
    data = means[mix] + 0.5 * torch.randn(n_samples, 2)  # σ=0.5

    # 스코어 네트워크 초기화
    score_net = SimpleScoreNetwork(input_dim=2, hidden_dim=128)
    optimizer = torch.optim.Adam(score_net.parameters(), lr=1e-3)

    # 훈련 루프 (Denoising Score Matching 사용)
    print("=== Denoising Score Matching 훈련 ===")
    for epoch in range(500):
```

```
        # 미니배치 샘플링
        idx = torch.randint(0, n_samples, (64,))
        batch = data[idx]

        # DSM 손실 계산 및 역전파
        loss = denoising_score_matching_loss(score_net, batch, sigma
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 100 == 0:
            print(f"  Epoch {epoch+1:4d} | DSM Loss: {loss.item():.

    # 학습된 스코어 확인: 모드 중심에서 스코어 ≈ 0이어야 함
    test_points = torch.tensor([[2.0, 2.0], [-2.0, -2.0], [0.0, 0.0
    with torch.no_grad():
        scores = score_net(test_points)
    print("\n학습된 스코어 확인:")
    for pt, sc in zip(test_points, scores):
        print(f"  위치 ({pt[0]:.1f}, {pt[1]:.1f}) → "
              f"스코어 ({sc[0]:.3f}, {sc[1]:.3f}), "
              f"크기 = {sc.norm():.3f}")
```

> ⚠️ **실제 이미지에서의 스코어 네트워크**: 위 예시는 2D 데이터를 위한 간단한 MLP이다. 실제 이미지(수만 차원)에서는 **U-Net** 구조를 사용하며, 입력 이미지와 동일한 해상도의 스코어 맵을 출력한다. Skip connection이 픽셀 단위의 정밀한 스코어 추정에 핵심적인 역할을 한다.

2.6 Langevin Dynamics: 스코어로부터 샘플링

> 💡 **왜 별도의 샘플링 알고리즘이 필요한가?** 스코어 $s_\theta(x) \approx \nabla_x \log p(x)$를 학습했다고 하자. 이것은 "모든 위치에서의 오르막 방향"을 안다는 뜻이다. 하지만 오르막 방향을 안다고 해서 자동으로 이미지가 생성되는 것은 아니다. 어떤 **시작점**에서 출발하여, 스코어를 따라 **반복적으로 이동**하는 알고리즘이 필요하다. 이것이 바로 Langevin Dynamics이다. 물리학에서 유래한 이 방법은 열역학적 입자가 에너지 곡면 위에서 움직이는 것을 시뮬레이션하며, 통계학에서는 MCMC(Markov Chain Monte Carlo) 샘플링의 한 형태이다.

스코어를 학습했으면, 이제 이 스코어를 이용해 **이미지를 생성(샘플링)**해야 한다.

문제: 스코어는 확률 분포가 아니므로 직접 샘플링할 수 없다. 화살표(방향)만 알고 있을 뿐.

해결: **Langevin Dynamics** -- 스코어(화살표)를 따라 점진적으로 이동하여 데이터 영역에 도달

$$\boxed{x_{t+1} = x_t + \frac{\epsilon}{2}\nabla_x \log p(x_t) + \sqrt{\epsilon}\, z_t, \quad z_t \sim \mathcal{N}(0, I)}$$

이 수식의 유도 과정: Langevin Dynamics는 다음의 연속 시간 확률 미분방정식(SDE)을 이산화한 것이다:

$$dx = \nabla_x \log p(x)\, dt + \sqrt{2}\, dW_t$$

여기서 dW_t는 위너 과정(Wiener process, 브라운 운동)이다. 이를 시간 간격 $\Delta t = \epsilon/2$로 Euler-Maruyama 이산화하면:

$$x_{t+1} = x_t + \frac{\epsilon}{2}\nabla_x \log p(x_t) + \sqrt{\epsilon}\, z_t$$

각 항의 의미:

항	역할
x_t	현재 위치
$\frac{\epsilon}{2}\nabla_x \log p(x_t)$	**그래디언트 어센트 항**: 스코어 방향으로 이동 (확률 높은 곳으로). $\log p(x)$를 최대화하는 방향이다
$\sqrt{\epsilon}\, z_t$	**노이즈 항**: 랜덤 노이즈 (탐색의 다양성 확보). 스텝 크기 ϵ에 비례하여 스케일링된다
ϵ	스텝 크기. $\epsilon/2$와 $\sqrt{\epsilon}$의 비율이 중요 -- 이 특정 비율이 정상 분포(stationary distribution)가 $p(x)$가 되도록 보장한다

과정:

1. **랜덤 노이즈로 초기화**: $x_0 \sim \mathcal{N}(0, I)$ (텅 빈 우주에서 출발)
2. **반복 이동**: 스코어 방향으로 조금씩 이동 (중력에 이끌려 행성으로 접근)
3. **수렴**: 충분히 반복하면 $p_{\text{data}}(x)$로부터의 샘플에 도달

> $\epsilon \to 0$이고 스텝 수 $T \to \infty$이면, x_T의 분포가 $p_{\text{data}}(x)$에 수렴한다는 것이 이론적으로 보장됨

> ⚠️ **노이즈 항이 왜 반드시 필요한가 -- 깊은 이해**: Langevin Dynamics에서 $\sqrt{\epsilon}z_t$ 노이즈 항이 없으면 단순한 gradient ascent가 되어 가장 가까운 모드(극대점)에만 수렴한다. 구체적 예를 들자: 데이터가 두 개의 가우시안 모드(예: 고양이와 개)로 구성되어 있다고 하자. 노이즈 없이 gradient ascent만 하면, 초기 위치에서 가까운 모드(예: 고양이)에만 수렴하고 다른 모드(개)는 절대 방문하지 않는다. 노이즈가 있으면, 가끔 "언덕을 넘어" 다른 모드로 이동할 수 있다. 충분한 시간이 지나면, 각 모드를 **그 모드의 확률 질량(probability mass)에 비례하는 빈도로** 방문하게 된다. 이것이 MCMC 이론의 핵심 결과이며, Langevin Dynamics가 $p_{\text{data}}(x)$에서의 **정확한 샘플링**을 보장하는 이유이다. 노이즈는 "실수"가 아니라 **이론적으로 필수적인 탐색 메커니즘**이다.

PyTorch 구현: Langevin Dynamics Sampling Loop

아래는 학습된 스코어 네트워크를 이용하여 Langevin Dynamics로 샘플을 생성하는 구현이다:

```python
import torch
import torch.nn as nn

@torch.no_grad()
def langevin_dynamics_sampling(
    score_net,
    initial_samples,
    step_size=0.01,
    n_steps=1000,
    noise_scale=1.0,
    return_trajectory=False,
):
    """
    Langevin Dynamics 샘플링: 스코어를 따라 노이즈에서 데이터 분포로 이

    업데이트 규칙:
    x_{t+1} = x_t + (ε/2) * s_θ(x_t) + √ε * z_t,  z_t ~ N(0, I)

    Args:
        score_net: 학습된 스코어 네트워크 s_θ
        initial_samples: 초기 노이즈 샘플 x_0 ~ N(0, I), shape = (B,
        step_size: 스텝 크기 ε (작을수록 정밀하지만 느림)
        n_steps: 총 반복 횟수 T
        noise_scale: 노이즈 스케일 조절 (1.0이 표준)
        return_trajectory: True면 모든 중간 단계를 반환 (시각화용)
    Returns:
        samples: 최종 샘플 또는 전체 궤적
    """
    x = initial_samples.clone()
    trajectory = [x.clone()] if return_trajectory else None

    for t in range(n_steps):
        # 현재 위치에서 스코어(기울기) 계산
        score = score_net(x)  # s_θ(x_t), shape = (B, D)

        # 랜덤 노이즈 샘플링
        noise = torch.randn_like(x)  # z_t ~ N(0, I)

        # Langevin 업데이트
        # x_{t+1} = x_t + (ε/2) * s_θ(x_t) + √ε * z_t
        x = x + (step_size / 2) * score + noise_scale * (step_size

        if return_trajectory:
            trajectory.append(x.clone())

    if return_trajectory:
        return torch.stack(trajectory, dim=0)  # (T+1, B, D)
    return x


@torch.no_grad()
def annealed_langevin_dynamics(
```

```
    score_net,
    initial_samples,
    sigmas,
    steps_per_sigma=100,
    step_size_factor=0.5,
    return_trajectory=False,
):
    """
    Annealed Langevin Dynamics: 다중 노이즈 스케일을 사용한 개선된 샘플

    큰 노이즈(σ₁)부터 시작하여 점차 작은 노이즈(σ_L)로 줄여가며 샘플링.
    각 노이즈 레벨에서 여러 스텝의 Langevin Dynamics 수행.

    핵심: 큰 σ에서는 먼 곳에서도 데이터 방향을 찾고,
          작은 σ에서는 정밀한 디테일을 완성

    σ₁ > σ₂ > ... > σ_L ≈ 0

    Args:
        score_net: 학습된 스코어 네트워크 s_θ(x, σ) (σ를 조건으로 받는
        initial_samples: 초기 노이즈 x_0 ~ N(0, I)
        sigmas: 노이즈 스케일 시퀀스 [σ₁, σ₂, ..., σ_L] (내림차순)
        steps_per_sigma: 각 노이즈 레벨에서의 Langevin 스텝 수
        step_size_factor: 스텝 크기 계수 (ε = factor * σ² / σ_L²)
        return_trajectory: True면 전체 궤적 반환
    Returns:
        samples: 최종 생성 샘플 또는 궤적
    """
    x = initial_samples.clone()
    trajectory = [x.clone()] if return_trajectory else None

    for i, sigma in enumerate(sigmas):
        # 각 σ 레벨에 맞는 스텝 크기 설정
        # σ가 클 때는 큰 스텝, 작을 때는 작은 스텝
        step_size = step_size_factor * (sigma / sigmas[-1]) ** 2

        for t in range(steps_per_sigma):
            # 현재 노이즈 레벨 σ를 조건으로 전달
            # score_net이 σ를 조건으로 받는 경우:
            sigma_tensor = torch.full(
                (x.shape[0], 1), sigma, device=x.device
            )
            score = score_net(torch.cat([x, sigma_tensor], dim=-1))

            # Langevin 업데이트
            noise = torch.randn_like(x)
            x = x + (step_size / 2) * score + (step_size ** 0.5) * n

            if return_trajectory:
                trajectory.append(x.clone())

        # 각 레벨 완료 시 진행 상황 출력 (선택적)
```

```
        # 각 레벨 완료 시 진행 상황 출력 (선택적)
        # print(f"  σ = {sigma:.4f} 완료 (레벨 {i+1}/{len(sigmas)})"

    if return_trajectory:
        return torch.stack(trajectory, dim=0)
    return x


class ConditionalScoreNetwork(nn.Module):
    """
    노이즈 레벨 σ를 조건으로 받는 스코어 네트워크
    Annealed Langevin Dynamics에서 사용

    s_θ(x, σ) ≈ ▽_x log p_σ(x)
    """
    def __init__(self, data_dim=2, hidden_dim=128):
        super().__init__()
        # 입력: 데이터 x (data_dim) + 노이즈 레벨 σ (1) = data_dim +
        self.net = nn.Sequential(
            nn.Linear(data_dim + 1, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, data_dim),  # 출력: 스코어 (data_d
        )

    def forward(self, x_with_sigma):
        """
        Args:
            x_with_sigma: [x, σ] 연결된 텐서, shape = (B, data_dim +
        Returns:
            score: 스코어 벡터, shape = (B, data_dim)
        """
        return self.net(x_with_sigma)


# === 전체 파이프라인 예시: 학습 + 샘플링 ===
if __name__ == "__main__":
    torch.manual_seed(42)
    data_dim = 2

    # 1. 2D 가우시안 혼합 데이터 생성
    n_samples = 2000
    mix = torch.randint(0, 3, (n_samples,))  # 3개 모드
    means = torch.tensor([[3.0, 0.0], [-1.5, 2.6], [-1.5, -2.6]])
    data = means[mix] + 0.3 * torch.randn(n_samples, 2)

    # 2. 조건부 스코어 네트워크 학습 (다중 노이즈 레벨)
    score_net = ConditionalScoreNetwork(data_dim=2, hidden_dim=256)
    optimizer = torch.optim.Adam(score_net.parameters(), lr=1e-3)
```

```python
# 노이즈 스케일 시퀀스: 큰 것부터 작은 것까지
sigmas = torch.tensor([2.0, 1.0, 0.5, 0.25, 0.1, 0.05])

print("=== 다중 노이즈 스케일 Denoising Score Matching 훈련 ===")
for epoch in range(2000):
    # 미니배치 샘플링
    idx = torch.randint(0, n_samples, (128,))
    batch = data[idx]  # (128, 2)

    # 랜덤 노이즈 레벨 선택
    sigma_idx = torch.randint(0, len(sigmas), (128,))
    sigma = sigmas[sigma_idx].unsqueeze(1)  # (128, 1)

    # 노이즈 추가
    noise = torch.randn_like(batch)
    x_noisy = batch + sigma * noise

    # 스코어 예측
    x_input = torch.cat([x_noisy, sigma], dim=-1)  # (128, 3)
    score_pred = score_net(x_input)  # (128, 2)

    # DSM 정답: -(x_noisy - batch) / σ² = -noise / σ
    target = -noise / sigma

    # MSE 손실
    loss = 0.5 * torch.mean(torch.sum((score_pred - target) ** 2

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 500 == 0:
        print(f"  Epoch {epoch+1:4d} | Loss: {loss.item():.4f}"

# 3. Annealed Langevin Dynamics로 새로운 샘플 생성
print("\n=== Annealed Langevin Dynamics 샘플링 ===")
n_gen = 500
initial_noise = torch.randn(n_gen, 2) * 3.0  # 넓게 퍼진 초기 노

generated_samples = annealed_langevin_dynamics(
    score_net=score_net,
    initial_samples=initial_noise,
    sigmas=sigmas.tolist(),
    steps_per_sigma=100,
    step_size_factor=0.5,
)

print(f"생성된 샘플 shape: {generated_samples.shape}")  # (500, 
print(f"생성 샘플 평균: ({generated_samples[:, 0].mean():.2f}, "
      f"{generated_samples[:, 1].mean():.2f})")
print(f"생성 샘플 표준편차: ({generated_samples[:, 0].std():.2f},
```

```
            f"{generated_samples[:, 1].std():.2f})")

    # 모드별 클러스터링으로 생성 품질 확인
    for i, mean in enumerate(means):
        dist = torch.norm(generated_samples - mean, dim=-1)
        count = (dist < 1.0).sum().item()
        print(f"  모드 {i+1} ({mean[0]:.1f}, {mean[1]:.1f}) 근처 샘플
              f"{count}/{n_gen}")
```

2.7 데이터 희소 영역 문제와 Noise Perturbation

> 💡 **왜 이론적으로 완벽한 스코어 매칭 + Langevin Dynamics가 실제로는 잘 안 되는가?** 지금까지의 이론은 아름답다: 스코어를 학습하고 Langevin Dynamics로 샘플링하면 된다. 하지만 실전에서는 심각한 문제가 있다. 자연 이미지의 데이터 분포 $p_{\text{data}}(x)$는 196,608차원 공간의 극히 얇은 **저차원 매니폴드(manifold)** 위에 집중되어 있다. 전체 공간의 99.999...%는 데이터가 전혀 없는 빈 공간이다. 이 빈 공간에서의 스코어 학습은 훈련 데이터가 없으므로 사실상 랜덤이다. 그런데 Langevin Dynamics의 시작점 $x_0 \sim \mathcal{N}(0, I)$은 거의 확실하게 이 빈 공간에 있다! 이것이 이론과 실전의 결정적 간극이다.

문제

데이터가 많은 영역: 스코어가 **정확하게** 학습됨 (훈련 데이터가 풍부) 데이터가 없는 영역: 스코어가 **부정확** (학습 데이터가 없으므로 거의 랜덤)

이를 수학적으로 표현하면: 스코어 매칭 로스 $\mathbb{E}_{x \sim p_{\text{data}}}[\cdot]$는 $p_{\text{data}}(x) > 0$인 영역에서만 기대값을 취한다. $p_{\text{data}}(x) \approx 0$인 영역에서는 학습 신호가 전혀 없으므로, 뉴럴넷 $s_\theta(x)$의 출력은 초기화에 의존하는 임의의 값이 된다.

이로 인한 결과: 랜덤 초기 위치에서 출발하면 데이터 영역까지 도달할 **가이드가 없음** (우주 빈 공간에서 중력이 없음)

> ⚠️ **이 문제가 중요한 이유**: Langevin Dynamics는 $x_0 \sim \mathcal{N}(0, I)$에서 출발하는데, 이 초기 위치는 거의 확실하게 데이터가 없는 영역이다. 스코어가 부정확한 영역에서 시작하므로, 잘못된 방향으로 이동하거나 어디로도 수렴하지 못할 수 있다. 이것이 단순한 Score Matching + Langevin Dynamics만으로는 고품질 샘플을 생성하기 어려운 근본 이유이다.

해결: 다중 노이즈 스케일 (Multi-Scale Noise Perturbation)

> 💡 **핵심 아이디어: 노이즈로 빈 공간을 채우기** 데이터 x에 가우시안 노이즈 $\sigma \cdot n$을 추가하면, 노이즈가 추가된 분포 $q_\sigma(\tilde{x})$는 원래 분포보다 **넓게 퍼진다**. σ가 충분히 크면, 빈 공간에서도 $q_\sigma(\tilde{x}) > 0$이 되므로 스코어 학습이 가능해진다. 이것은 "무에서 유를 창조"하는 것이 아니라, 노이즈가 추가된 분포의 스코어를 학습하는 것이다. 큰 σ의 스코어는 부정확하지만 "대략 어느 방향에 데이터가 있는가"라는 정보는 제공하고, 작은 σ의 스코어는 데이터 근처에서만 정확하지만 그 영역에서는 매우 정밀하다.

큰 노이즈를 섞으면 확률 분포가 **넓게 퍼진다** → 빈 공간에서도 스코어가 존재 → 대략적인 방향 제공

$$\tilde{x} = x + \sigma \cdot n, \quad n \sim \mathcal{N}(0, I)$$

- σ 크면: 분포가 넓게 퍼짐, 먼 곳에서도 끌어당길 수 있지만 **부정확** (원본 이미지의 디테일이 노이즈에 묻힘)
- σ 작으면: 원래 분포에 가까움, **정확**하지만 먼 곳에는 영향 없음 (데이터 매니폴드 근처에서만 유효)

점진적 노이즈 감소 전략:

$$\sigma_1 > \sigma_2 > \sigma_3 > \cdots > \sigma_L \approx 0$$

```
[단계 1] σ₁ (큰 노이즈): 대략적인 방향으로 이동 → 데이터 근처 도달
[단계 2] σ₂ (중간 노이즈): 더 정확한 방향으로 이동 → 모드 근처 도달
  ...
[단계 L] σ_L ≈ 0 (노이즈 거의 없음): 미세 조정 → 깨끗한 이미지 완성
```

> DDPM(Denoising Diffusion Probabilistic Models)은 이 과정을 **1,000단계**에 걸쳐 수행

> 💡 **왜 여러 개의 노이즈 레벨이 필요한가? (Noise Conditional Score Networks)** 하나의 노이즈 레벨만 사용하면 어떻게 될까? σ가 크면 먼 곳에서 데이터를 찾을 수 있지만, 최종 생성 이미지가 흐릿하다(큰 노이즈 스케일의 스코어는 세밀한 디테일을 모르므로). σ가 작으면 디테일은 정확하지만, Langevin Dynamics가 데이터 영역을 찾지 못한다. **단일 노이즈 레벨은 "대략적 구조"와 "세밀한 디테일"을 동시에 만족시킬 수 없다.** 다중 노이즈 레벨은 이 딜레마를 해결한다: 큰 σ에서 시작하여 전체 구조를 잡고, 점차 작은 σ로 전환하면서 디테일을 추가한다. 이것은 화가가 먼저 대략적 스케치를 그리고, 그 위에 세부 묘사를 추가하는 과정과 동일하다.

뉴럴넷은 하나

노이즈 스케일 σ는 **조건 입력**으로 뉴럴넷에 제공:

$$s_\theta(x, \sigma) \approx \nabla_x \log p_\sigma(x)$$

같은 뉴럴넷 s_θ를 다른 σ 값으로 반복 호출하여, 노이즈가 큰 상태에서 작은 상태로 점진적으로 이미지를 정제.

> 💡 **Score-Based Model과 Diffusion Model의 깊은 연결** 여기서 핵심적인 통찰이 나온다: 이 "큰 노이즈에서 작은 노이즈로의 점진적 제거" 과정은 DDPM(Denoising Diffusion Probabilistic Models)의 **역방향 과정(reverse process)**과 수학적으로 동등하다. DDPM에서는 forward process로 데이터에 점진적으로 노이즈를 추가하고($x_0 \to x_1 \to \cdots \to x_T$), reverse process로 노이즈를 점진적으로 제거한다($x_T \to x_{T-1} \to \cdots \to x_0$). Score-Based Model에서는

> 다양한 σ 레벨에서의 스코어를 학습하고, Annealed Langevin Dynamics로 큰 σ에서 작은 σ로 진행한다. Yang Song et al. (2021)은 이 두 접근법이 모두 하나의 **확률 미분방정식(SDE)** 프레임워크로 통합됨을 보였다:
>
> Forward SDE: $dx = f(x,t)dt + g(t)dW_t$ (노이즈 추가)
>
> Reverse SDE: $dx = [f(x,t) - g(t)^2 \nabla_x \log p_t(x)]dt + g(t)d\bar{W}_t$ (노이즈 제거)
>
> reverse SDE에서 $\nabla_x \log p_t(x)$가 바로 시간 t에서의 **스코어 함수**이다. 따라서 스코어를 학습하면 reverse SDE를 시뮬레이션하여 샘플링할 수 있고, 이것이 Score-Based Model = Diffusion Model인 이유이다. DDPM의 노이즈 예측 ϵ_θ와 스코어 예측 s_θ의 관계는: $s_\theta(x_t, t) = -\epsilon_\theta(x_t, t)/\sqrt{1-\bar{\alpha}_t}$이다.

2.8 생성 과정 요약

```
x₀ ~ N(0, I)     [완전한 노이즈 — 잡음 이미지]
    ↓ s_θ(x, σ₁) 호출 + Langevin step
x₁                [대략적 형태 출현]
    ↓ s_θ(x, σ₂) 호출 + Langevin step
x₂                [구조가 명확해짐]
    ↓  ...
x_T               [깨끗한 이미지 완성]
```

시간이 지남에 따라: 잡음 → 대략적 형태 → 세부 구조 → 깨끗한 이미지

Score-Based Generation의 전체 파이프라인을 DALL-E와 대비하여 정리하면:

단계	DALL-E (자기회귀)	Score-Based Model (디퓨전)
표현	이산 토큰 (코드북 인덱스)	연속 벡터 (픽셀 또는 잠재 변수)
생성 방식	토큰을 하나씩 순차 생성	전체 이미지를 동시에 점진적 정제
조건 부여	텍스트 토큰과 concat	스코어 네트워크에 조건 입력
다양성	소프트맥스 샘플링	Langevin 노이즈
속도	1,024번 순차 forward pass	$T \times L$번 forward pass (T: 노이즈 레벨, L: 레벨당 스텝)
장점	언어 모델 기술 직접 활용	아키텍처 제약 적음, 이미지 품질 우수
단점	느린 생성, 이산화 손실	수백~수천 스텝 필요 (이후 가속 연구로 해결)

Score-Based Model이 이후 Diffusion Model로 발전하면서, 이미지 생성의 품질과 유연성에서 자기회귀 방식을 크게 앞서게 되었다. 이것이 DALL-E 2/3가 자기회귀를 버리고 디퓨전으로 전환한 이유이다.

---

3. 핵심 수식 총정리

이 섹션은 강의에서 등장한 모든 핵심 수식을 한눈에 정리한다. 각 수식이 **왜 그런 형태**인지 한 줄 요약을 함께 제공한다.

DALL-E 훈련 손실

$$\mathcal{L} = -\sum_{i=1}^{M} \log p_\theta(t_i \mid t_{<i}) - \sum_{j=1}^{1024} \log p_\theta(s_j \mid t_{1:M}, s_{<j})$$

첫째 항: 텍스트 자기회귀 예측 (보조 손실). 둘째 항: 이미지 자기회귀 예측 (핵심 손실). 둘 다 Cross-Entropy이므로 음의 로그 확률의 합이다.

스코어 함수

$$s(x) = \nabla_x \log p(x)$$

확률이 높아지는 방향을 가리키는 벡터. 정규화 상수 Z가 미분으로 사라지므로, $\nabla_x \log \tilde{p}(x)$만 알면 된다.

가우시안의 스코어

$$s(x) = -\frac{x-\mu}{\sigma^2}$$

항상 평균 μ를 향해 끌어당긴다. σ가 작으면(분산이 좁으면) 끌어당기는 힘이 강하다.

Denoising Score Matching (실전에서 가장 많이 사용)

$$\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{x,n}\left[\left\| s_\theta(x+\sigma n) + \frac{n}{\sigma} \right\|^2\right], \quad n \sim \mathcal{N}(0, I)$$

야코비안 계산 불필요. 노이즈가 추가된 데이터에서의 조건부 스코어 $-n/\sigma$를 정답으로 사용한다.

스코어 매칭 목적 함수 (원본, Fisher Divergence)

$$\mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}_{x\sim p_{\mathrm{data}}}\left[\| s_\theta(x) - \nabla_x \log p_{\mathrm{data}}(x) \|_2^2\right]$$

이상적 형태. 진짜 스코어를 모르므로 직접 사용 불가.

스코어 매칭 목적 함수 (부분적분 후, 계산 가능한 형태)

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\text{tr}(\nabla_x s_\theta(x)) + \frac{1}{2} \| s_\theta(x) \|_2^2 \right]$$

부분 적분으로 진짜 스코어를 제거. 단, 야코비안 트레이스 계산이 고차원에서 비실용적.

Langevin Dynamics

$$x_{t+1} = x_t + \frac{\epsilon}{2} s_\theta(x_t) + \sqrt{\epsilon}\, z_t, \quad z_t \sim \mathcal{N}(0, I)$$

그래디언트 어센트(확률 높은 곳으로 이동) + 노이즈(다양한 모드 탐색). $\epsilon/2$와 $\sqrt{\epsilon}$의 비율이 $p(x)$로의 수렴을 보장한다.

---

4. 이후 학습 로드맵

강의에서 "이런 생성 모델들은 피지컬 AI를 하든 뭘 하든 **필수**"라고 강조했다. 이 강의 시리즈에서 다루지 못한, 이후에 반드시 공부해야 할 핵심 주제들을 정리한다.

4.1 반드시 공부해야 할 핵심 모델들

(1) DDPM (Denoising Diffusion Probabilistic Models)

> 📄 **논문**: Ho et al., "*Denoising Diffusion Probabilistic Models*", NeurIPS 2020

Score-Based Model의 실용적 구현. 이번 강의에서 배운 스코어 개념의 **직접적 연장선**.

핵심 개념:

- **Forward Process**: 깨끗한 이미지에 점진적으로 노이즈를 추가 ($T = 1000$ 스텝)
- **Reverse Process**: 노이즈 이미지에서 점진적으로 노이즈를 제거하여 깨끗한 이미지 생성
- 뉴럴넷이 학습하는 것: 각 스텝에서 추가된 **노이즈를 예측** (ϵ-prediction)
- U-Net 구조 사용

핵심 수식 (참고):

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\| \epsilon - \epsilon_\theta(x_t, t) \|^2 \right]$$

DDPM과 Score-Based Model의 관계: **수학적으로 동등**하다. 스코어를 학습하는 것과 노이즈를 예측하는 것은 변환 관계에 있다:

$$s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}$$

> 💡 **왜 이 관계가 성립하는가?** Forward process에서 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$이므로, $q(x_t|x_0)$의 스코어는 $\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1-\bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$이다. 즉, 노이즈 ϵ을 예측하는 것과 스코어를 예측하는 것은 단순한 스케일링 관계이다. 이번 강의에서 배운 Score-Based Model의 이론이 DDPM의 수학적 토대라는 것을 명확히 인식하는 것이 중요하다.

(2) Latent Diffusion Model (LDM) / Stable Diffusion

> 📄 **논문**: Rombach et al., "*High-Resolution Image Synthesis with Latent Diffusion Models*", CVPR 2022
>
> 저자: VQ-GAN의 Esser, Rombach, Ommer

핵심 개념:

- **픽셀 공간에서 직접 디퓨전하지 않음** → 계산량 문제
- VQ-GAN/VAE의 **잠재 공간(latent space)**에서 디퓨전 수행
- 텍스트 조건: CLIP 텍스트 인코더로 처리 → Cross-Attention으로 U-Net에 주입

```
텍스트 → CLIP Encoder → 조건 임베딩
                                ↓ (Cross-Attention)
노이즈 z_T → [U-Net Denoising xT] → 깨끗한 잠재 표현 z₀ → [VAE Decode
```

> 💡 **이번 강의에서 배운 것과의 연결**: Latent Diffusion은 이 강의 시리즈에서 배운 두 가지를 정확히 결합한 것이다. VQ-GAN(강의 4)의 인코더/디코더로 이미지를 잠재 공간으로 압축하고, 그 잠재 공간에서 Score-Based Model(강의 5)의 디퓨전 프로세스를 수행한다. 픽셀 공간($256 \times 256 \times 3$)에서 디퓨전하면 계산량이 막대하지만, 잠재 공간($32 \times 32 \times 4$ 등)에서 디퓨전하면 약 48배 작은 공간에서 동일한 과정을 수행하므로 실용적이 된다. 이것이 "Stable Diffusion"이라는 이름으로 오픈소스화되어 생성 AI의 대중화를 이끌었다.

(3) Flow Matching

> 📄 **논문**: Lipman et al., "*Flow Matching for Generative Modeling*", ICLR 2023

최신 생성 모델의 또 다른 축. 강의에서 "Diffusion, Flow Matching이 중요하다"고 언급.

핵심 개념:

- 노이즈 분포 → 데이터 분포 사이의 **연속적 경로(flow)**를 학습
- ODE(상미분방정식)를 사용하여 샘플링
- Diffusion 대비: 더 적은 스텝으로 생성 가능, 학습이 단순

Stable Diffusion 3, FLUX 등 최신 모델이 Flow Matching 기반.

(4) Classifier-Free Guidance (CFG)

> 📄 **논문**: Ho & Salimans, "*Classifier-Free Diffusion Guidance*", NeurIPS Workshop 2022

조건부 생성 품질을 크게 향상시키는 핵심 기법:

$$\tilde{\epsilon}_\theta(x_t, c) = \epsilon_\theta(x_t, \varnothing) + w \cdot (\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing))$$

- c: 조건 (텍스트 등), $\varnothing$: 무조건
- $w > 1$: guidance scale (높을수록 조건에 충실, 다양성 감소)
- 사실상 모든 최신 Text-to-Image 모델이 사용

4.2 기반 지식으로 필요한 것들

(5) CLIP (Contrastive Language-Image Pre-training)

> 📄 **논문**: Radford et al., "*Learning Transferable Visual Models From Natural Language Supervision*", ICML 2021

텍스트-이미지 생성 모델의 **조건 인코더**로 거의 필수:

- 텍스트와 이미지를 공유 임베딩 공간에 매핑
- DALL-E 2, Stable Diffusion, Imagen 등에서 텍스트 인코더로 사용

(6) DALL-E 2 / DALL-E 3

버전	이미지 생성 방식	텍스트 인코더
DALL-E 1 (이번 강의)	자기회귀 (트랜스포머)	BPE 토크나이저
DALL-E 2	**Diffusion**	**CLIP**
DALL-E 3	Diffusion (개선)	CLIP + 캡션 개선

(7) SDE/ODE 기반 통합 프레임워크

> 📄 **논문**: Yang Song et al., "*Score-Based Generative Modeling through Stochastic Differential Equations*", ICLR 2021

Score-Based Model과 DDPM을 **확률 미분방정식(SDE)**으로 통합한 프레임워크. 이론적 이해를 위해 중요.

4.3 응용 분야

(8) 비디오 생성

- Sora (OpenAI), Runway Gen-3 등
- 시간 축이 추가된 3D U-Net + Diffusion/Flow Matching

- Temporal Attention 추가

(9) 3D 생성

- DreamFusion, Magic3D 등
- Score Distillation Sampling (SDS): 2D 디퓨전 모델로 3D 최적화

(10) Physical AI / Robotics

강의에서 강조한 영역:

- 액션 생성: Diffusion Policy (로봇 행동을 디퓨전으로 생성)
- World Model: 비디오 예측 모델로 환경 시뮬레이션
- 이미테이션 러닝만으로는 부족 → AI 전반의 이해 필요

4.4 추천 학습 순서

```
[현재 위치: 이 강의 시리즈 완료]
        │
        ▼
    ① DDPM (Score-Based의 실용 버전)
        │
        ▼
    ② CLIP (텍스트-이미지 연결의 핵심)
        │
        ▼
    ③ Latent Diffusion / Stable Diffusion (실제 사용되는 모델)
        │
        ▼
    ④ Classifier-Free Guidance (조건부 생성 핵심 기법)
        │
        ▼
    ⑤ Flow Matching (최신 트렌드, SD3/FLUX)
        │
        ▼
    ⑥ SDE/ODE 통합 프레임워크 (이론적 완성)
        │
        ▼
    ⑦ 응용: 비디오 생성 / 3D 생성 / Physical AI
```

---

5. 참고문헌 (References)

DALL-E 및 텍스트-이미지 생성

1. Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., & Sutskever, I. (2021). *Zero-Shot Text-to-Image Generation*. Proceedings of the

38th International Conference on Machine Learning (ICML). arXiv:2102.12092

2. Ding, M., Yang, Z., Hong, W., Zheng, W., Zhou, C., Yin, D., Lin, J., Zou, X., Shao, Z., Yang, H., & Tang, J. (2021). *CogView: Mastering Text-to-Image Generation via Transformers*. Advances in Neural Information Processing Systems (NeurIPS). arXiv:2105.13290

3. Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). *Hierarchical Text-Conditional Image Generation with CLIP Latents* (DALL-E 2). arXiv:2204.06125

4. Betker, J., Goh, G., Jing, L., et al. (2023). *Improving Image Generation with Better Captions* (DALL-E 3). OpenAI Technical Report.

Score-Based 및 Diffusion 모델

5. Song, Y. & Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. Advances in Neural Information Processing Systems (NeurIPS). arXiv:1907.05600

6. Song, Y. & Ermon, S. (2020). *Improved Techniques for Training Score-Based Generative Models*. Advances in Neural Information Processing Systems (NeurIPS). arXiv:2006.09011

7. Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. Advances in Neural Information Processing Systems (NeurIPS). arXiv:2006.11239

8. Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations*. International Conference on Learning Representations (ICLR). arXiv:2011.13456

9. Ho, J. & Salimans, T. (2022). *Classifier-Free Diffusion Guidance*. NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications. arXiv:2207.12598

Latent Diffusion 및 최신 모델

10. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2112.10752

11. Esser, P., Kulal, S., Blattmann, A., Entezari, R., Muller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., et al. (2024). *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis* (Stable Diffusion 3). arXiv:2403.03206

12. Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow Matching for Generative Modeling*. International Conference on Learning Representations (ICLR). arXiv:2210.02747

기반 기술

13. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). *Learning Transferable Visual Models From Natural Language Supervision* (CLIP). International Conference on Machine Learning (ICML). arXiv:2103.00020

14. van den Oord, A., Vinyals, O., & Kavukcuoglu, K. (2017). *Neural Discrete Representation Learning* (VQ-VAE). Advances in Neural Information Processing Systems (NeurIPS). arXiv:1711.00937

15. Esser, P., Rombach, R., & Ommer, B. (2021). *Taming Transformers for High-Resolution Image Synthesis* (VQ-GAN). IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2012.09841

Score Matching 이론

16. Hyvarinen, A. (2005). *Estimation of Non-Normalized Statistical Models by Score Matching*. Journal of Machine Learning Research, 6, 695-709.

17. Song, Y., Garg, S., Shi, J., & Ermon, S. (2020). *Sliced Score Matching: A Scalable Approach to Density and Score Estimation*. Uncertainty in Artificial Intelligence (UAI). arXiv:1905.07088

응용 분야

18. Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., & Song, S. (2023). *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*. Robotics: Science and Systems (RSS). arXiv:2303.04137

19. Poole, B., Jain, A., Barron, J. T., & Mildenhall, B. (2023). *DreamFusion: Text-to-3D using 2D Diffusion*. International Conference on Learning Representations (ICLR). arXiv:2209.14988

> *이 강의 노트는 생성 모델의 핵심 개념을 다루며, DALL-E의 자기회귀 텍스트-이미지 생성에서 Score-Based Model의 이론적 기초까지를 포괄한다. 이후 DDPM, Latent Diffusion, Flow Matching 등 현대 생성 모델의 학습을 위한 토대가 된다.*